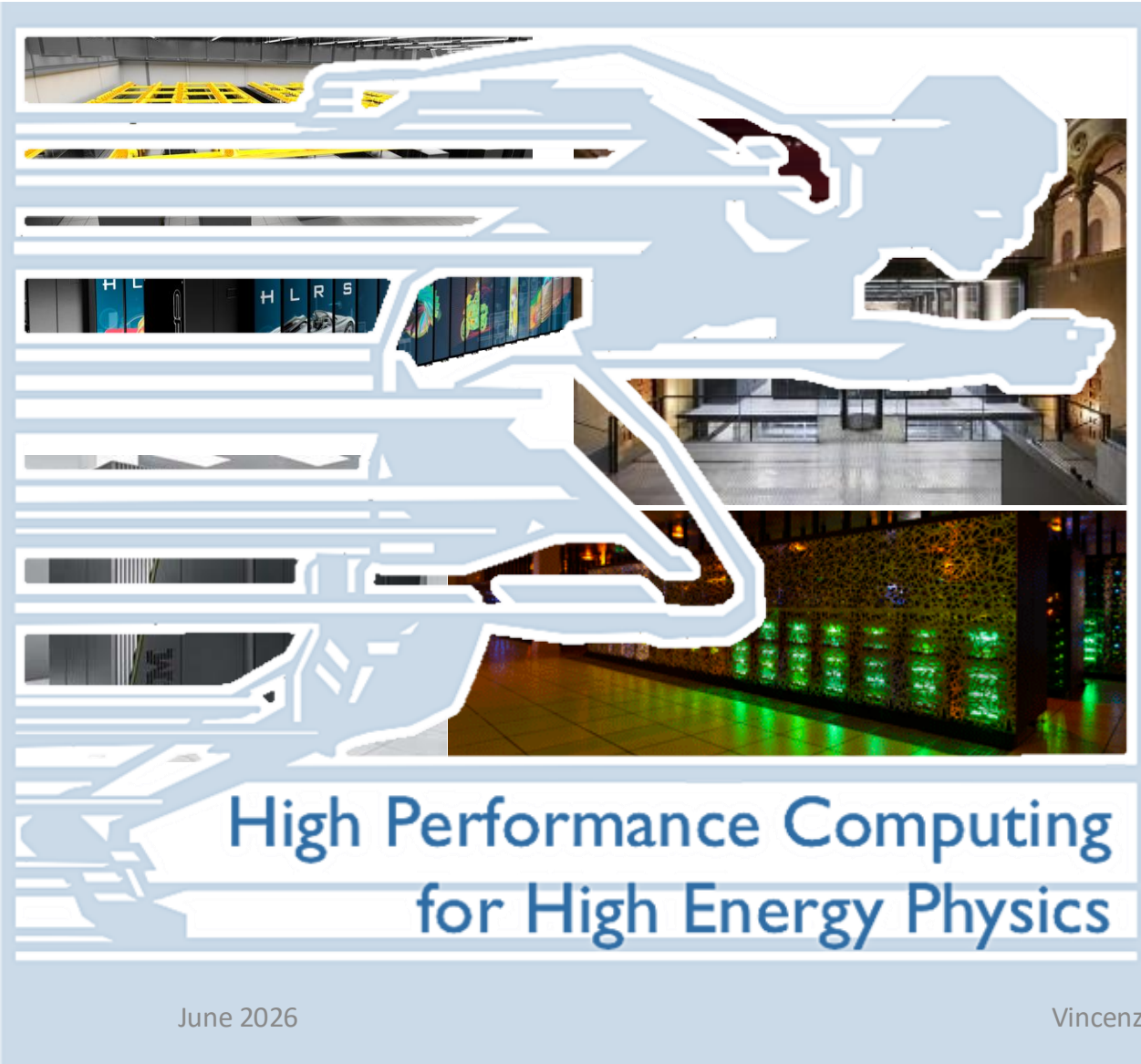


FloatingPoint Arithmetic for Scientists: Precision, Accuracy, Efficiency



Vincenzo Innocente

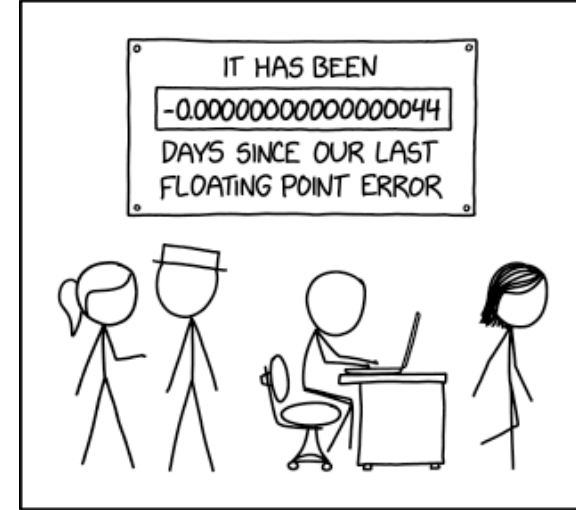
CERN, June 2026

Definitions

What is a Floating Point?

https://en.wikipedia.org/wiki/Floating-point_arithmetic

- Science is based on Measurements
- The result of a Measurement is a Rational Number
 - $M_H = 125.04 \pm 0.12 \text{ GeV} = 2.2290 \cdot 10^{-25} \pm 2.1 \cdot 10^{-28} \text{ kg}$
- “Floating Point” is a way to represent Rational Numbers by a fixed number of digits (precision) multiplied by a “base” with an exponent
 - A form of “Scientific Notation”



$31 \cdot 10^{-1}$ is a FP in base 10 with “precision” 2 (# significant digits) printed in decimal

FP are usually “normalized” : the first digit is always not zero

1.1001001000011111011011 is a FP in base 2 with precision 24 printed in binary

normalized is a 1 followed by a string of 23bits. Easier to read if printed in hexadecimal: $1.921fb6 \cdot 2^{+1}$. NB in hex last digit is always even

Some Trivial properties of FP

- Each FP has a previous and a next
 - Previous of $31 \cdot 10^{-1}$ is $30 \cdot 10^{-1}$, next is $32 \cdot 10^{-1}$
 - Previous of $1.921fb6 \cdot 2^{+1}$ is $1.921fb4 \cdot 2^{+1}$, next is $1.921fb8 \cdot 2^{+1}$
 - The difference between two consecutive FP is called ULP (unit in last place)
- For each value of the exponent there is a finite number of FP
 - $10 \cdot 10^e - 99 \cdot 10^e$
 - $1.000000 \cdot 2^e - 1.fffffe \cdot 2^e$
(called “binade”)

Precision and Rounding

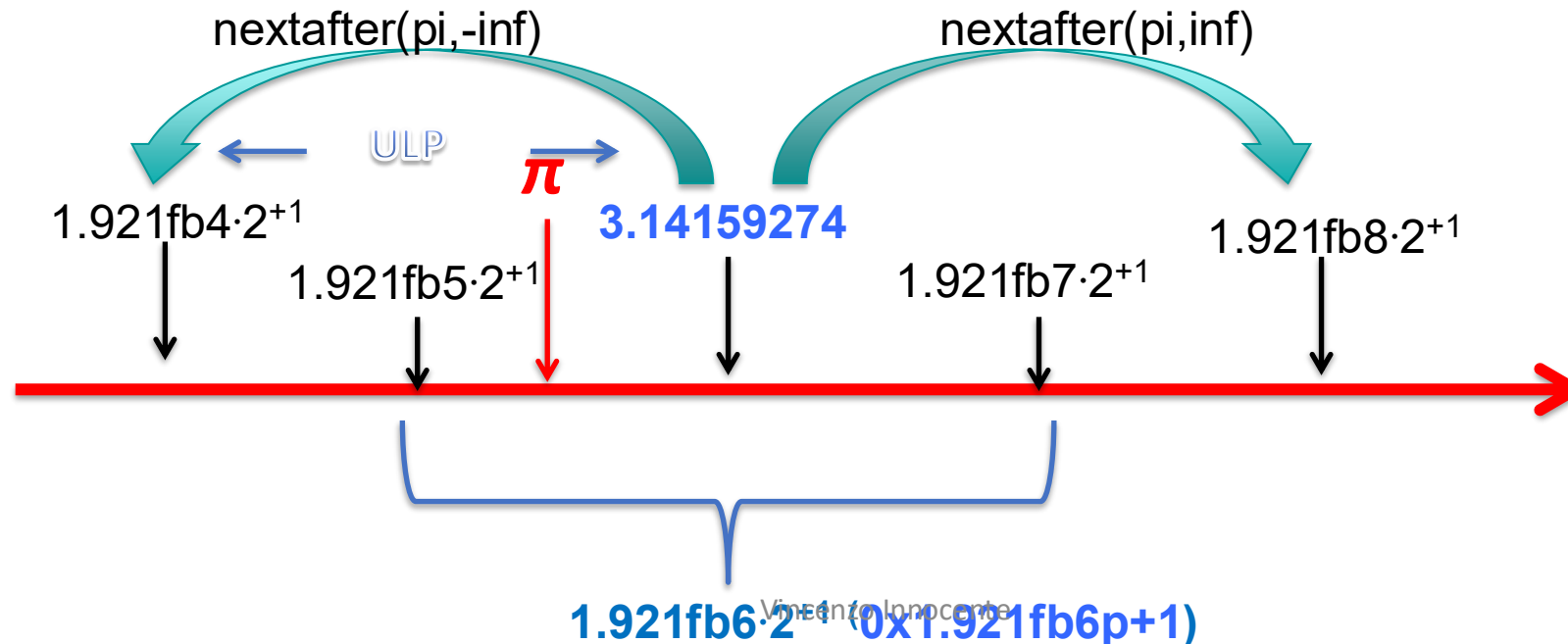
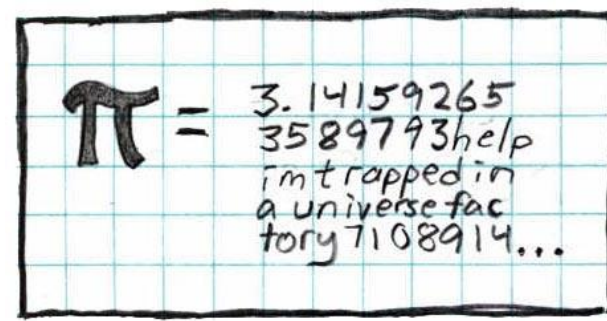
The result of a measurement is 3.1415 and its accuracy, say, 0.1.

This result needs to be rounded. Default is to round to “nearest”

3.1415 $\rightarrow$ 3.1 (3.1515 $\rightarrow$ 3.2)

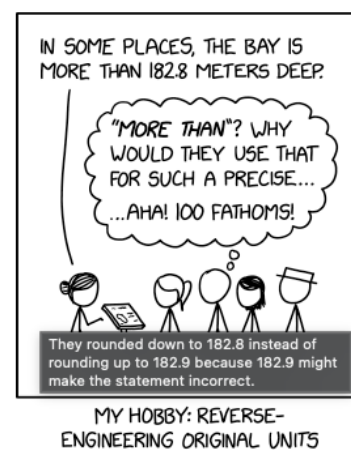
$31 \cdot 10^{-1}$ represents all numbers in the interval $[30.5 \cdot 10^{-1}, 31.5 \cdot 10^{-1}]$

$1.921\text{fb}6 \cdot 2^{+1}$ represents all numbers in the interval $[1.921\text{fb}5 \cdot 2^{+1}, 1.921\text{fb}7 \cdot 2^{+1}]$



Rounding Algorithms

- The standard defines five rounding algorithms.
 - The first two round to a nearest value; the others are called directed roundings:
- Roundings to nearest
 - **Round to nearest, ties to even – rounds to the nearest value;** if the number falls midway it is rounded to the nearest value with an even (zero) least significant bit, which occurs 50% of the time; **this is the default algorithm for binary floating-point** and the recommended default for decimal
 - Round to nearest, ties away from zero – rounds to the nearest value; if the number falls midway it is rounded to the nearest value above (for positive numbers) or below (for negative numbers)
- Directed roundings
 - Round toward 0 – directed rounding towards zero (also known as truncation).
 - Round toward $+\infty$ – directed rounding towards positive infinity (also known as rounding up or ceiling).
 - Round toward $-\infty$ – directed rounding towards negative infinity (also known as rounding down or floor).
- A “function” is **correctly rounded** if the result corresponds to the rounded true value
- A “function” is **faithfully rounded** if the result corresponds to one of the two FP closest to the true value



FP on a Computer

- FP are well suited to perform arithmetic on a computer
 - Since 1985 FP representations and operations are standardized in IEEE754 (https://en.wikipedia.org/wiki/IEEE_754)

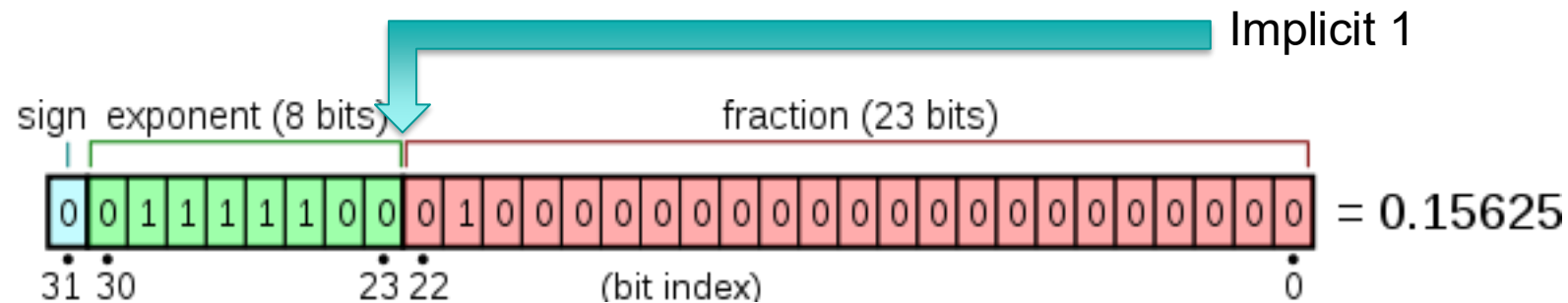
The standard defines:

- *arithmetic formats*: sets of **binary** and **decimal** floating-point data, which consist of finite numbers (including **signed zeros** and **subnormal numbers**), **infinities**, and special "not a number" values (**NaNs**)
- *interchange formats*: encodings (bit strings) that may be used to exchange floating-point data in an efficient and compact form
- *rounding rules*: properties to be satisfied when rounding numbers during arithmetic and conversions
- *operations*: arithmetic and other operations (such as **trigonometric functions**) on arithmetic formats
- *exception handling*: indications of exceptional conditions (such as **division by zero**, overflow, etc.)

FP binary representations in IEEE754

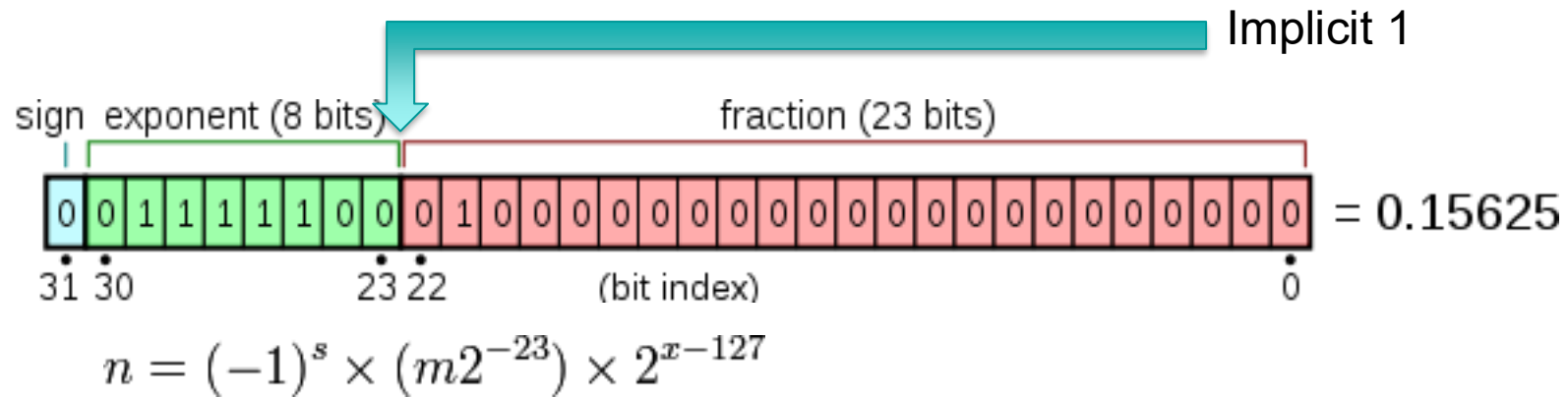
Format	Bits for the encoding			
	Sign	Exponent	Significand	Total
Half (binary16)	1	5	10	16
Single (binary32)	1	8	23	32
Double (binary64)	1	11	52	64
x86 extended	1	15	64	80
Quadruple (binary128)	1	15	112	128
Octuple (binary256)	1	19	236	256

Exponent bias	Bits precision	Number of decimal digits
15	11	~3.3
127	24	~7.2
1023	53	~15.9
16383	64	~19.2
16383	113	~34.0
262143	237	~71.3



$$n = (-1)^s \times (m2^{-23}) \times 2^{x-127}$$

IEEE 754 representation of single precision FP



Exponent	significand zero	significand non-zero	Equation
00 _H	zero, -0	subnormal numbers	$(-1)^{\text{signbits}} \times 2^{-126} \times 0.\text{significandbits}$
01 _H , ..., FE _H	normalized value	normalized value	$(-1)^{\text{signbits}} \times 2^{\text{exponentbits}-127} \times 1.\text{significandbits}$
FF _H	±infinity	NaN (quiet,signalling)	

examples

		sign	exponent	significant
0.00000000e+00	0x0p+0	0	00000000	00000000000000000000000000000000
5.00000000e-01	0x1p-1	0	01111111	00000000000000000000000000000000
1.00000000e+00	0x1p+0	0	01111111	00000000000000000000000000000000
1.50000000e+00	0x1.8p+0	0	01111111	10000000000000000000000000000000
2.00000000e+00	0x1p+1	0	10000000	00000000000000000000000000000000
2.50000000e+00	0x1.4p+1	0	10000000	01000000000000000000000000000000
3.00000000e+00	0x1.8p+1	0	10000000	10000000000000000000000000000000
8.00000000e+00	0x1p+3	0	10000010	00000000000000000000000000000000
3.14159274e+00	0x1.921fb6p+1	0	10000000	10010010000111111011011
1.00000001e-01	0x1.99999ap-4	0	01111011	100110011001100110011001101
1.17549435e-38	0x1p-126	0	00000001	00000000000000000000000000000000
3.40282347e+38	0x1.fffffep+127	0	11111111	01111111111111111111111111111111
nan	nan	1	11111111	10000000000000000000000000000000
-inf	-inf	1	11111111	00000000000000000000000000000000
2.93873588e-39	0x1p-128	0	00000000	01000000000000000000000000000000

C++ interlude

```
#include<iostream>
#include<iomanip>
#include<cmath>
#include<limits>
// std style (prior to C++20)

template<typename T>
void print(T x) {
    std::cout << std::hexfloat << x << ' ' // exact binary representation
               << std::scientific << std::setprecision(8) << x << ' ' // 8 decimal digits
               << std::defaultfloat << x << std::endl; // 6 decimal digits for float
}

// integer representation // https://godbolt.org/z/9x1hxMhhP
#include<cstring>
#include<bitset>
int f2i(float x) { int i; memcpy(&i,&x,sizeof(float)); return i;} // compiler will optimize
// use bitset to print in binary (bit by bit)
int ix = f2i(x);
std::cout << std::bitset<32>(ix) << std::endl; ; // bit by bit
int e = ((ix >> 23) & 255) - 127; //exponent
ix &= 0x7FFFFFFF; //mantissa (aka significand) (w/o leading "1")
```

Limits

- Everything you want to know about an arithmetic type is in
 - `std::numeric_limits<T>`
 - see http://www.cplusplus.com/reference/limits/numeric_limits/

```
#include <iostream>    // std::cout
#include <limits>       // std::numeric_limits
int main () {
    std::cout << std::boolalpha; // << std::hexfloat;
    std::cout << "Minimum value: " << std::numeric_limits<float>::min() << '\n';
    std::cout << "Maximum value: " << std::numeric_limits<float>::max() << '\n';
    std::cout << "Is signed: " << std::numeric_limits<float>::is_signed << '\n';
    std::cout << "significand bits: " << std::numeric_limits<float>::digits << '\n';
    std::cout << "precision: " << std::numeric_limits<float>::epsilon() << '\n';
    std::cout << "has infinity: " << std::numeric_limits<float>::has_infinity << '\n';
    return 0;
}
```

Fixed width floating-point types (since C++23)

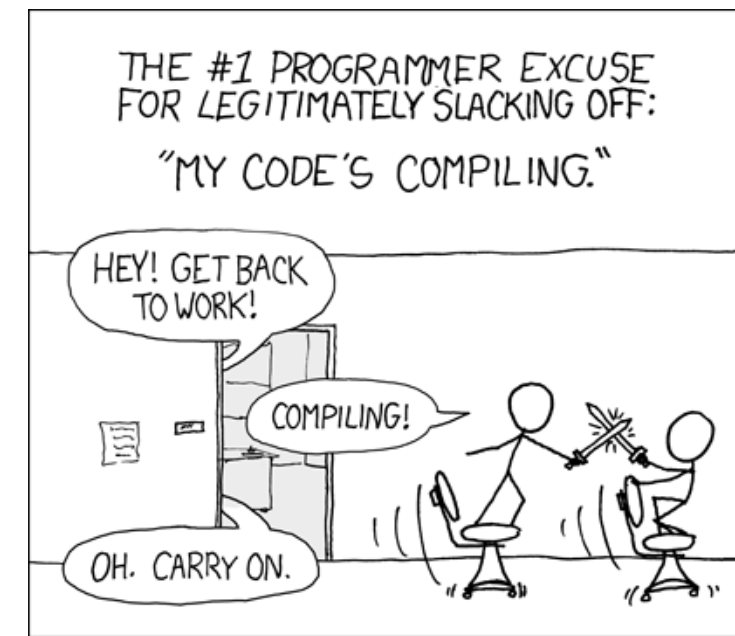
<https://en.cppreference.com/w/cpp/types/floating-point.html>

Types Defined in header <stdfloat>	Literal suffix	Predefined macro	C language type	Type properties			
				bits of storage	bits of precision	bits of exponent	max exponent
float16_t	f16 or F16	__STDCPP_FLOAT16_T__	_Float16	16	11	5	15
float32_t	f32 or F32	__STDCPP_FLOAT32_T__	_Float32	32	24	8	127
float64_t	f64 or F64	__STDCPP_FLOAT64_T__	_Float64	64	53	11	1023
float128_t	f128 or F128	__STDCPP_FLOAT128_T__	_Float128	128	113	15	16383
bf16_t	bf16 or BF16	__STDCPP_BFLOAT16_T__	(N/A)	16	8	8	127

Python interlude

- Python intrinsic “float” is actually double precision
 - `float.hex()` and `float.fromhex()` allows to convert to/from hexadecimal string
- **numpy** defines `float16`, `float32`, `float64` (and `float128` on x86_64)
 - [https://numpy.org/doc/2.3/reference/arrays.scalars.html - floating-point-types](https://numpy.org/doc/2.3/reference/arrays.scalars.html#floating-point-types)
- **decimal** is a module providing FP in base 10 of arbitrary precision
 - <https://docs.python.org/3/library/decimal.html>

EXERCISES

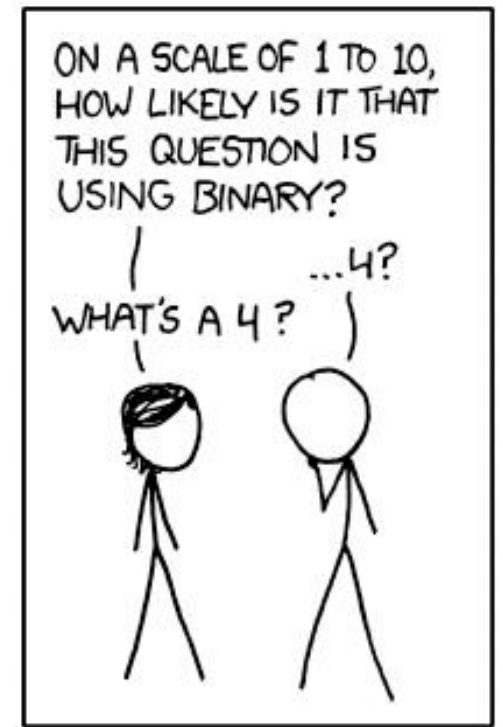


Either in Python or C++

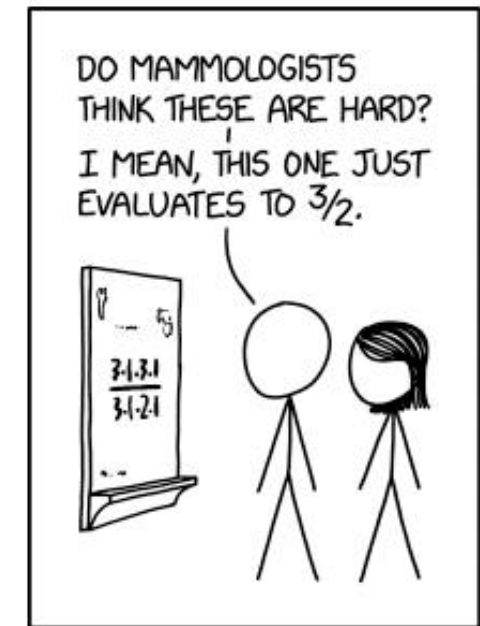
For C++ one can use godbolt (<https://godbolt.org/z/15GTzW17a>)

For Python either a python3 shell on your PC, or a jupyter notebook or the browser shell in <https://numpy.org/>

- Count the number of FP in a binade
 - Verify that it is the same for two different binades
- Compute the ULP for some FPs
 - How it depends on the binade?
- Convert some number (0.1, e, pi, 31.5, ...) between various representations (including decimal)
 - See what happens and explain it



Arithmetic



MATHEMATICIANS ENCOUNTER
DENTAL FORMULAS

FP Arithmetic

- There are two basic operations: sum and multiplication
 - $a+b$ and $a \times b$ symbolize the exact sum and multiplication between a and b
 - $a \oplus b$ and $a \otimes b$ symbolize the rounded sum and multiplication between FPs a and b
 - $a+b$ and $a*b$ are the corresponding operations in code
- We will use decimal precision 2 for all examples to allow mental arithmetic in a finite time
 - $42+3.7 = \dots$
 - $42 \oplus 3.7 = \dots$
 - $42 \oplus 3.3 = \dots$, $42 \oplus 3.5 = \dots$, $43 \oplus 3.5 = \dots$
 $42 \oplus 3.3 \oplus 3.4 = \dots$, $3.4 \oplus 3.3 \oplus 42 = \dots$
 $42 \oplus 3.3 \oplus -45 = \dots$

(use python decimal to verify results)
- FP Arithmetic is not associative and can lead to loss of precision up to catastrophic cancellations
 - $x + \varepsilon - x \neq \varepsilon$ (can easily be 0 or ∞)

Classical example of loss of precision due to cancellation

For example, consider the quadratic equation:

$$ax^2 + bx + c = 0,$$

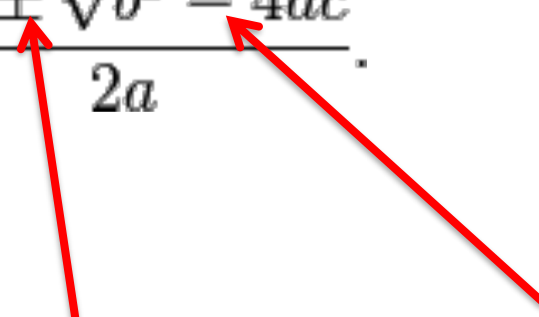
with the two exact solutions:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

We know better solution

$$x_1 = \frac{-b - \operatorname{sgn}(b) \sqrt{b^2 - 4ac}}{2a},$$

$$x_2 = \frac{2c}{-b - \operatorname{sgn}(b) \sqrt{b^2 - 4ac}} = \frac{c}{ax_1}.$$



Two sources of cancellation

https://en.wikipedia.org/wiki/Loss_of_significance

A first theorem

- The difference of two number of the same binade is precise
 - $99-11 = 88$, $42-37 = 5$, $3.1-2.7=0.4$
 - $0x1.921fb6p+1 - 0x1.001e72p+1 = 0x1.240288p+0$
 - $0x1.921fb6p+1 - 0x1.5bf0a8p+1 = 0x1.b17870p-2$
 - $\pi - e = 0x1.b17864p-2$
- If $2b > a > b/2 > 0$ then $a \ominus b = a-b$ (is precise)
(https://en.wikipedia.org/wiki/Sterbenz_lemma)
 - $12-7.4 = 4.6$ (precise)
 - $18-7.4 = 10.6 \Rightarrow 18 \ominus 7.4 = 11$ (not precise)
 - $0x1.333334p-2 \ominus 0x1.99999ap-3 = 0x1.99999ap-4$ (precise)
 - $0x1.333334p-2 \ominus 0x1.99999ap-4 = 0x1.99999cp-3$ (not precise)

Augmented Arithmetic

- If $a > b > 0 \Rightarrow a + b < 2a$
- $c = a \oplus b$
- $t = c \ominus a$ is exact
- $e = b \ominus t$ is exact
- $a + b = c + e$
 - e is the error of $a \oplus b$
- This algorithm is known as “fastTwoSum” and (with small modifications) proposed as AugmentedSum in next IEEE754 revision
- A similar algorithm exists for multiplication as well

Example of augmented sum

<https://godbolt.org/z/nq5Po1Gdz>

- $42 \oplus 3.7 = 46$
 - $46 \ominus 42 = 4$
 - $3.7 \ominus 4 = -0.3$
 - $46 - 0.3 = 42 + 3.7$ (*q.e.d.*)
 - In short: $\text{asum}(42, 3.7) = 46, -0.3$
-
- $42 \oplus 3.3 \oplus 3.4 = 48$ while $42 + 3.3 + 3.4 = 48.7$
 - $\text{asum}(42, 3.3) = 45, 0.3$
 - $\text{asum}(45, 3.4) = 48, 0.4$
 - $\text{asum}(48, 0.3 + 0.4) = 49, -0.3$

Does it help for catastrophic cancellation?

- $42 \oplus 3.3 \oplus -45 = 0$ while $42 + 3.3 - 45 = 0.3$
 - Catastrophic cancellation
- $\text{asum}(42, 3.3) = 45, 0.3$
- $\text{asum}(45, -45) = 0.0, 0.0$
- $\text{asum}(0.0, 0.3 + 0.0) = 0.3, 0.0$

Compensating algorithms

- Kahan summation
 - https://en.wikipedia.org/wiki/Kahan_summation_algorithm
 - with excruciating details <https://arxiv.org/html/2602.19452v1> - LST2

```
T kahanSum(T const * input, size_t n) {  
    T sum = input[0];  
    T t = 0.0;  
    for (size_t i = 1; i < n; ++i) {  
        auto y = input[i] - t;  
        auto s = sum + y;  
        t = (s - sum) - y;  
        sum = s;  
    }  
    return sum;  
}
```

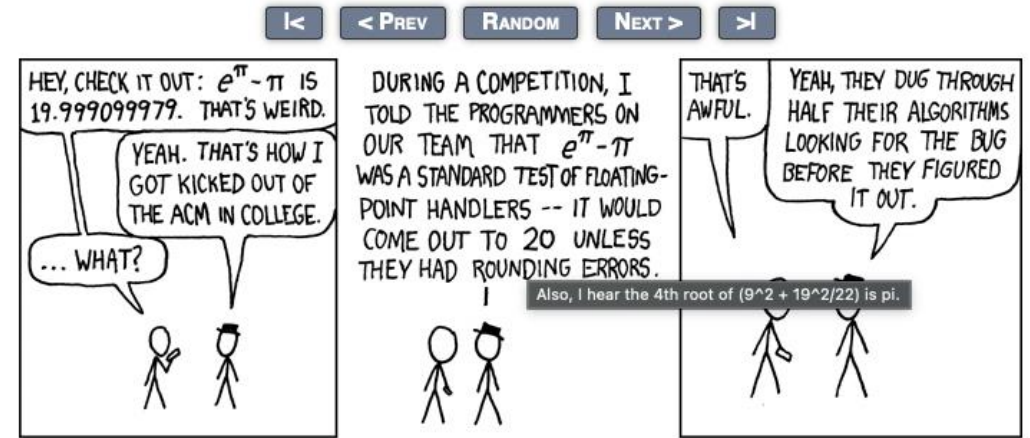
Improved algorithms exist:

- differently handling of various magnitude ranges (“big”, “medium”, “tiny”)
- (partial) sorting
- pairwise recursion
- optimization for parallelism

FMA: Fused Multiple Add

- Also known as MAC (multiply accumulate) instruction on AMD
- `fma(a,b,c)` computes the correctly rounded result of $a*b+c$
 - It improves speed and precision in many computation such as linear algebra
 - No standard on how to “contract” expression (depends on compiler)
 - $a*b+c*d$, $a+b*c+d$, $x*=a;x+=b$, etc
 - It clearly introduces asymmetries
 - for instance $a*b+c*d \neq c*d+a*b$
- Kahan comes to the rescue again with this algorithm

```
T kahanDet(T a, T b, T c, T d) { // a*d-b*c
    using std::fma;
    auto w = b*c;
    auto e = fma(-b, c, w); // error on w: (w,-e) = augmentedMultiply(b,c)
    auto f = fma(a, d, -w);
    return (f+e);
}
```



EXERCISES

Either in Python or C++

For C++ one can use godbolt (<https://godbolt.org/z/d1e18j3jK>)

For Python either a python3 shell on your PC, or a jupyter notebook or the browser shell in <https://numpy.org/>

- Sum $T(0.1)$ 10000 times in binary32, binary64, decimal2, decimal5
 - Google for “patriot failure”
- Sum 10^9 positive random numbers (in range 10^{-n} , 10^{+n})
 - Typical in unbinned log-likelihood
- Compute the average age of all humans on earth
- Compute π as the integral of $4/(1+x^2)$ between 0 and 1 (use *float*, *double*, *simd*, *int*, *mixed*)
 - (credit: Tim Mattson)

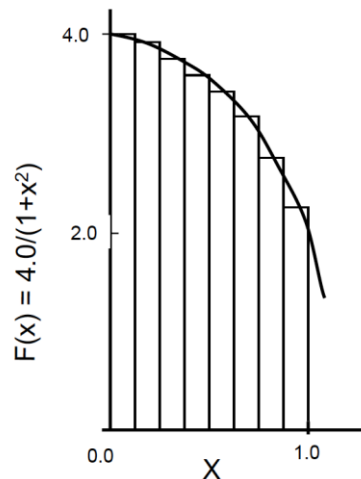
<https://godbolt.org/z/h6bEa8GY1>

```
static long num_steps = 100000;
double step;
int main ()
{
    int i;    double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;

    for (i=0; i< num_steps; i++){
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
    }
    pi = step * sum;
```

Numerical integration



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^N F(x_i) \Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i .

- Solve the equation $x^2 - 2bx + 1 = 0$ using the “high-school” formula
 - What is the smallest value of b (single precision) for which one of the solution becomes 0?
 - What is instead the correct result? (use only single precision computations)

Random Number interlude

```
#include <iostream>
#include <iomanip>
#include <random>
#include <algorithm>
#include <cmath>
int main()
{
    std::random_device rd; // to obtain a seed for the random number engine
    std::mt19937 gen(rd()); // Standard mersenne_twister_engine seeded with rd()
    std::uniform_real_distribution<float> dis(-1.0, 1.0);
    size_t N = 40*1000*1000;
    float * x = new float[N];
    for (int n = 0; n < N; ++n) {
        // Use dis to transform the random unsigned int generated by gen into a
        // float in [-1, 1). Each call to dis(gen) generates a new random float.
        x[n] = dis(gen);
    }
    // sort in absolute value
    std::sort(x,x+N,[](float a, float b) { return std::abs(a) < std::abs(b);});
    // print first and last 40
    for (int i=0; i<40; ++i) std::cout << x[i] << ' '; std::cout << '\n';
    for (int i=0; i<40; ++i) std::cout << x[N-1-i] << ' '; std::cout << '\n'; std::cout << '\n';
    std::cout << std::hexfloat;
    for (int i=0; i<40; ++i) std::cout << x[i] << ' '; std::cout << '\n';
    for (int i=0; i<40; ++i) std::cout << x[N-1-i] << ' '; std::cout << '\n';

    delete [] x;
    return 0;
}
```

```
int getRandomNumber()
{
    return 4; // chosen by fair dice roll.
             // guaranteed to be random.
}
```

```
0 -5.96046e-08 1.19209e-07 -1.19209e-07 1.19209e-07 -1.19209e-
07 1.19209e-07 1.19209e-07 -1.78814e-07 2.38419e-07 -
2.38419e-07 -3.57628e-07 3.57628e-07 3.57628e-07 -4.17233e-07
-4.17233e-07 -4.17233e-07 4.76837e-07 4.76837e-07 4.76837e-07
5.96046e-07 -5.96046e-07 -6.55651e-07 7.15256e-07 -7.15256e-
07 -7.7486e-07 8.34465e-07 8.34465e-07 -8.34465e-07 8.34465e-
07 -8.9407e-07 -9.53674e-07 -9.53674e-07 -9.53674e-07
9.53674e-07 1.07288e-06 1.07288e-06 1.19209e-06 1.19209e-06
1.19209e-06
1 -1 1 1 1 -1 1 1 1 1 -1 -1 -1 -1 -1 1 1 0.999999 0.999999 -0.999999
-0.999999 -0.999999 -0.999999 -0.999999 0.999999 0.999999
0.999999 0.999999 -0.999999 -0.999999 -0.999999 0.999999
0.999999 -0.999999 -0.999999 -0.999999 0.999999 0.999999
0.999999 -0.999999
```

```
0x0p+0 -0x1p-24 0x1p-23 -0x1p-23 0x1p-23 -0x1p-23 0x1p-23
0x1p-23 -0x1.8p-23 0x1p-22 -0x1p-22 -0x1.8p-22 0x1.8p-22
0x1.8p-22 -0x1.cp-22 -0x1.cp-22 -0x1.cp-22 0x1p-21 0x1p-21 0x1p-
21 0x1.4p-21 -0x1.4p-21 -0x1.6p-21 0x1.8p-21 -0x1.8p-21 -0x1.ap-
21 0x1.cp-21 0x1.cp-21 -0x1.cp-21 0x1.cp-21 -0x1.ep-21 -0x1p-20 -
0x1p-20 -0x1p-20 0x1p-20 0x1.2p-20 0x1.2p-20 0x1.4p-20 0x1.4p-
20 0x1.4p-20
0x1p+0 -0x1p+0 0x1p+0 0x1p+0 0x1p+0 -0x1.ffffep-1 0x1.ffffcp-1
0x1.ffffcp-1 0x1.ffff8p-1 0x1.ffff4p-1 -0x1.ffff4p-1 -0x1.ffff2p-1 -
0x1.ffff2p-1 -0x1.ffffp-1 -0x1.ffffp-1 0x1.ffffp-1 0x1.ffffp-1
0x1.ffffecp-1 0x1.ffffecp-1 -0x1.ffffeap-1 -0x1.ffffeap-1 -0x1.ffffeap-1
-0x1.ffffeap-1 -0x1.ffffeap-1 0x1.ffffe8p-1 0x1.ffffe8p-1 0x1.ffffe8p-1
0x1.ffffe8p-1 -0x1.ffffe6p-1 -0x1.ffffe6p-1 -0x1.ffffe6p-1 0x1.ffffe4p-1
0x1.ffffe4p-1 -0x1.ffffe2p-1 -0x1.ffffe2p-1 -0x1.ffffep-1 0x1.ffffep-1
0x1.ffffep-1 0x1.ffffep-1 -0x1.ffffep-1
```

Uniform vs canonical

(for a detailed discussion see among others

<https://github.com/DiscreteLogarithm/canonical-random-float/tree/master>)

Uniform in range [a,b)

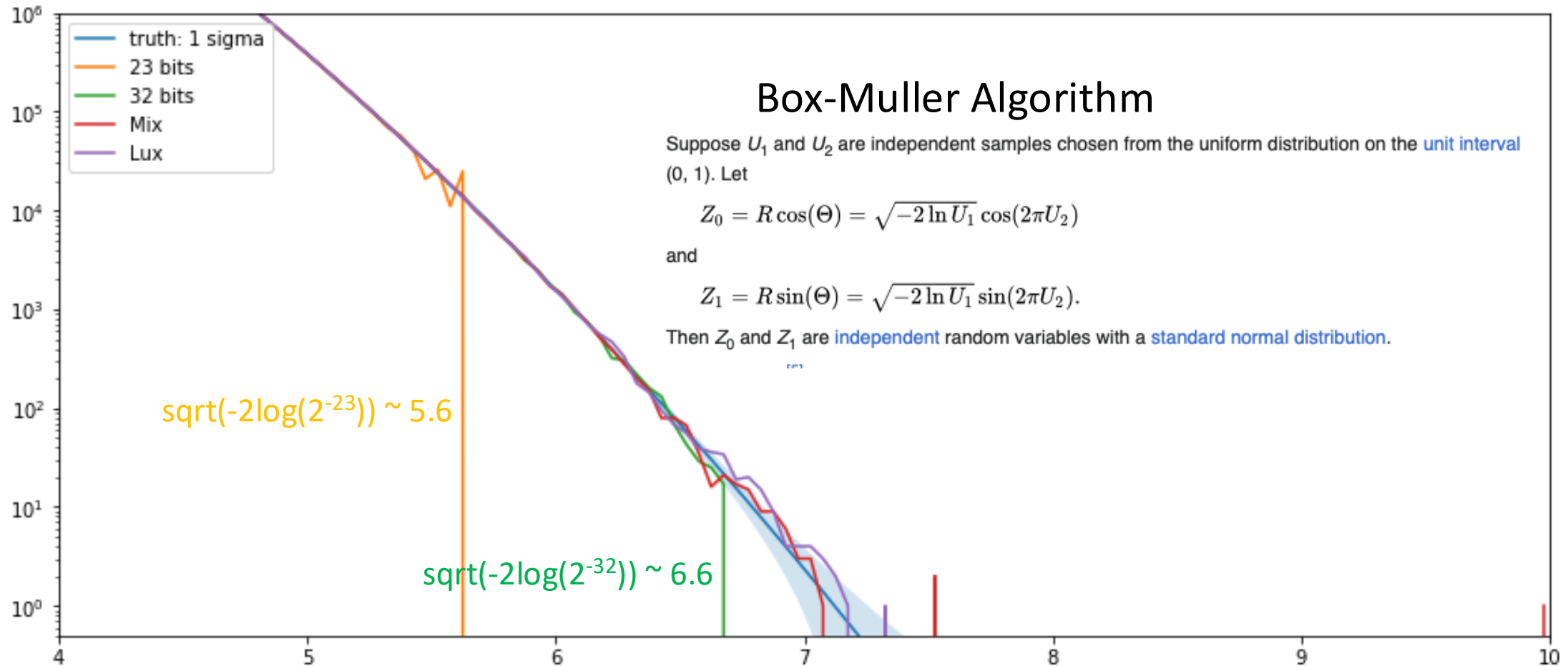
- Throw 32 coins: interpret as a positive integer N
- Compute $r = (\text{float}(N)/\text{float}(2^{32})) * (b-a) - a$

Canonical

- Throw 23 coins: set significand of r to them
- Throw one more coin: if 1 set exponent of r to -1: *uniform in $[0.5, 1)$ prob. $1/2$*
 - If 0 : Throw one more coin: if 1 set exponent of r to -2: *uniform in $[0.25, 0.5)$ prob. $1/4$*
 - If 0: Throw one more coin: if 1 set exponent of r to -3: *uniform in $[0.125, 0.25)$ prob. $1/8$*
 - If 0: Throw one more coin: if 1 set exponent of r to -4: *uniform in $[0.0625, 0.125)$ prob. $1/16$*
 - If 0:
- If symmetric interval around 0 needed : use last coin to choose sign

Gaussian Tail

- Generated $256 \cdot 10^9$ Normal distributed using Box-Muller
 - Mix: 23 bits for U2, 41 bits for U1
 - Lux: all-possible floats for both



Elementary Functions

Elementary functions

- IEEE754 defines a set of “additional operations” (aka elementary functions)
 - exp, log, power of various forms
 - trig, hyperbolic and their inverse (arc)
 - They are “required” to be correctly rounded
- C and C++ std define some more special functions
 - <https://en.cppreference.com/cpp/numeric/math>
 - https://en.wikipedia.org/wiki/C_mathematical_functions
- Each vendor/compiler provides its own implementation
 - A comparison of their accuracy is compiled by B.Gladman et al.
 - <https://members.loria.fr/PZimmermann/papers/accuracy.pdf>
- The most accurate implementation to date is from the “core-math” project
 - <https://core-math.gitlabpages.inria.fr/>

Library functions

library version	GNU libc 2.35	IML 2021.2.0	AMD 3.8	Newlib 4.2.0	OpenLibm 0.8.1	Musl 1.2.2	Apple 12.1	LLVM 13.0.0	CUDA 11.5.0	ROCm 4.5.2
acos	0.899	0.528	0.897	0.899	0.918	0.918	0.634	NA	1.69	1.47
acosh	2.01	0.501	0.504	2.01	2.01	NaN	0.502	NA	2.18	0.564
asin	0.898	0.528	0.781	0.926	0.743	0.743	0.634	NA	2.05	2.54
asinh	1.78	0.527	0.518	1.78	1.78	1.78	0.515	NA	1.78	0.573
atan	0.853	0.541	0.501	0.853	0.853	0.853	0.722	NA	2.06	2.10
atanh	1.73	0.507	0.506	1.73	1.73	1.73	0.511	NA	3.16	0.574
cbrt	0.969	0.520	0.548	3.56	0.500	0.500	0.500	NA	1.17	1.14
cos	0.561	0.548	0.729	2.91	0.501	0.501	0.862	0.776	1.52	1.61
cosh	1.89	0.506	8.39e6	2.51	1.36	1.03	0.589	NA	2.34	0.567
erf	0.968	0.507	0.968	0.968	0.943	0.968	0.501	NA	1.14	2.48
erfc	3.13	0.502	3.13	63.9	3.17	3.13	0.750	NA	4.49	3.33
exp	0.502	0.506	0.501	1.94e4	0.911	0.502	0.576	0.502	1.94	1.00
exp10	0.502	0.507	1.00	1.06	NA	3.88	0.580	NA	2.07	1.00
exp2	0.502	0.519	0.501	1.02	0.501	0.502	0.570	0.502	2.39	0.871
expm1	0.813	0.544	0.536	0.813	0.813	0.813	0.687	1.50	1.45	1.45

Maximal error (in bits) for single precision functions as implemented in various math library

- <https://members.loria.fr/PZimmermann/papers/accuracy.pdf>

Evaluating a function

for a recent review see <https://dl.acm.org/doi/pdf/10.1145/3747840>

Five steps

1. Handling special values, under/over-flows and invalid
 - Standard specify them all!
2. Range Reduction
3. Approximation
4. If required:
 - precision test and handling of “Table Maker’s Dilemma”
5. Back to full range

Range reduction

- exp, log, pow
 - exploit fp representation: $x = 1.m \cdot 2^n$
 - For $\log(x)$: $x = y \cdot 2^z$, with y in $[0.75, 1.5)$. $\log(x) = z \cdot \log(2) + \log(y)$
 - For $\exp(x)$: $x = k \cdot \log(2) + y$, with y in $[-\log(2)/2; \log(2)/2)$. $\exp(x) = 2^k \cdot \exp(y)$
 - $x^y = e^{y \cdot \log(x)}$
- sin, cos, tan, atan(2)
 - Reduce to $[0, \pi/4)$ (trivial for $f(\pi \cdot x)$ known as $\sin \pi i, \cos \pi i$ etc, tricky for $\sin(x)$ etc)
 - use trig identities to go back
- asin, acos, hyperbolic
 - approximate it in “ad-hoc” intervals (make sure continuity at edges)

Approximation

- Since 1964 the reference for function approximation is the *Abramowitz and Stegun*
 - Today replaced by the *digital library* <https://dlmf.nist.gov/>
- Elementary functions are usually approximated either by
 - shift-and-add algorithms (CORDIC <https://en.wikipedia.org/wiki/CORDIC>)
 - Today limited to FPGA
 - a polynomial
 - Chebyshev approximation satisfying the "superior" norm (max in an interval)
 - a rational function (ratio of two polynomials)
 - Pade' approximation
 - an iterative method such as Newton-Raphson

See also the short paper by J.M.Muller <https://hal.science/hal-02517784v1/file/approxfunctions.pdf>

$1/x$, $\sqrt{x}$, $1/\sqrt{x}$ aka $\text{rcp}(x)$, $\text{sqrt}(x)$, $\text{rsqrt}(x)$

- Few fast-converging Newton-Raphson iterations
 - Starting point is an approximation of $f(1.m)$

The major contribution of game industry to CS is the discovery of the “magic” fast $1/\sqrt{x}$ algorithm in quake3

```
float rsqrt(float x){  
    union {float f;int i;} tmp;  
    tmp.f = x;  
    tmp.i = 0x5f3759df - (tmp.i >> 1);  
    float y = tmp.f; // approximate  
    return y * (1.5f - 0.5f * x * y * y); // NR  
}
```

Real x86_64 code for $1/\sqrt{x}$

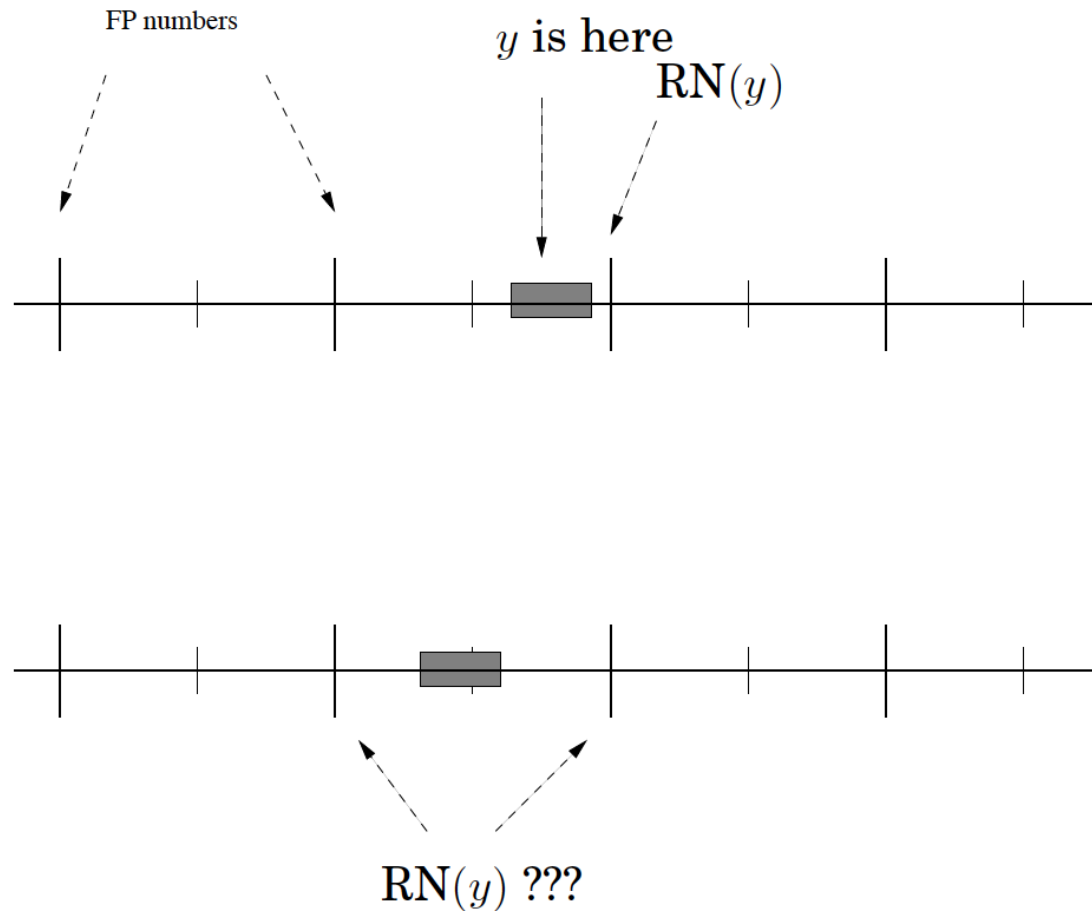
```
_mm_store_ss( &y, _mm_rsqrt_ss( _mm_load_ss( &x ) ) );  
return y * (1.5f - 0.5f * x * y * y); // One round of Newton's method
```

Real x86_64 code for $1/x$

```
_mm_store_ss( &y, _mm_rcp_ss( _mm_load_ss( &x ) ) );  
return (y+y) - x*y*y; // One round of Newton's method
```

https://en.wikipedia.org/wiki/Fast_inverse_square_root

Table Maker's Dilemma



to guarantee a given precision one needs to evaluate the error of the approximation " $y = f(x)$ "

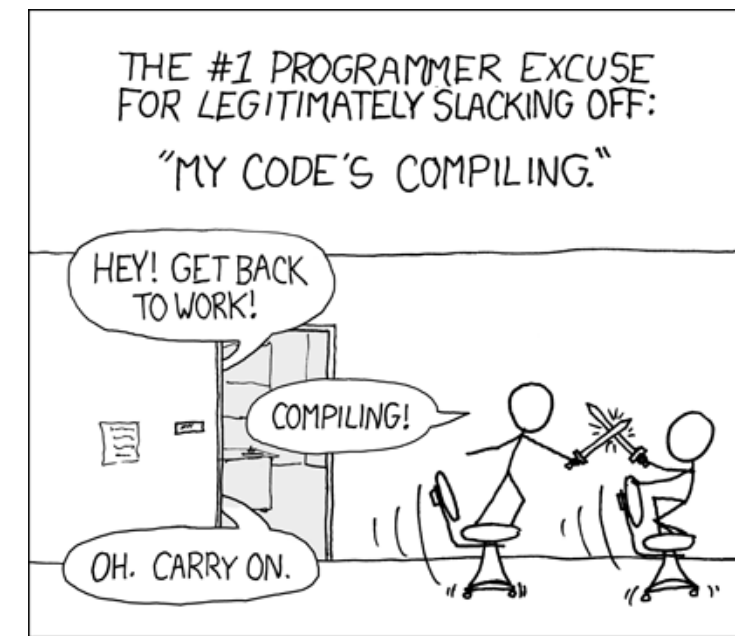
- From first principle
 - Or on the fly as seen before
1. The function need to be computed with at least one more digit (the round digit)
 2. In Round2Near: if the error interval includes the middle point an even more precise evaluation is required
- the round digit may be followed by a sequel of 0's or 1's (or 9's) and the TMD solution may be very expensive

Examples :

$\sin(0.40225342) = 0x1.\textcolor{red}{90e382}\text{ffff}7\text{bp-2}$

$\sin(0.47485355) = 0x1.\textcolor{red}{d42e63}000000\text{bp-2}$

EXERCISES



Either in Python or C++

For C++ one can use godbolt (<https://godbolt.org/z/d1e18j3jK>)

For Python either a python3 shell on your PC, or a jupyter notebook or the browser shell in <https://numpy.org/>

Polynomials

- Approximate $\sin(x)$ in $[0, \pi/4)$ with polynomials of increasing order
 - Taylor
 - Chebyshev
 - Pade
- Plot the absolute and relative error w/r/t a high precision result
- Use python mpmath module
<https://mpmath.readthedocs.io/en/latest/calculus/approximation.html>
- or sage, or mathematica...

- Test `rsqrt` precision in the interval $[0.5, 4)$
 - use “quake magic” and/or `x86_64` intrinsics plus multiple NR iterations
 - Try to reach faithful (or even correct) rounding.
- Compute $\sin(0x1.e64002p-2)$ using Taylor expansion:
 - How many terms are needed to reach faithful rounding in single precision?
 - How many more terms to reach correct rounding (in single precision)?
 - Use double precision to compute, then round to single

TMD in decimal

n	x	$\cos(x)$
4	1.149	0.409400000428...
5	1.6406	-0.69747000000219...
6	1.41162	0.15850500000180...
7	1.225910	0.33808970000005907...
8	1.9509449	-0.37105844000000012...

Table 1: Worst cases for n -digit mantissa, radix-10, floating-point numbers between 1 and 2, directed rounding modes and the cosine function.

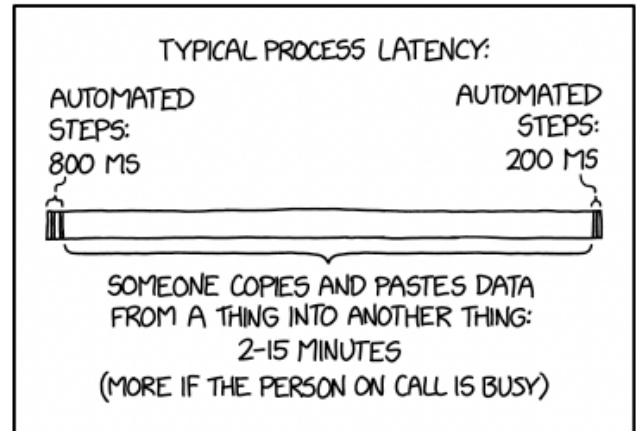
Use $\cos(x)$ approximation in decimal (taylor)

see <https://docs.python.org/3/library/decimal.html> - recipes

to “solve” TMD above (mind: direct rounding!)

LATENCY

[<](#) [< PREV](#) [RANDOM](#) [NEXT >](#) [>](#)



Precision, Accuracy, Speed

Time for FP optimization (?!?)

Once

- Removed all (unnecessary) Malloc
- Transformed all $\mathcal{O}(n^3)$ and $\mathcal{O}(n^2)$ algorithms to $\mathcal{O}(n \log(n))$ or even $\mathcal{O}(n)$
- Removed all (unnecessary) unpredictable branches (*read: virtual calls*)
- Improved Memory latency and bandwidth
- Solved all issues of correctness (as described in previous slides)

It's time for FP optimization!!!

For “anecdotes” see

<https://indico.cern.ch/event/1170924/contributions/4951098/attachments/2510594/4315064/HepSpec22.pdf>

Definitions

• Precision

- Numerical precision (in bits)
 - For an instrument (ruler!) typically “half a tick”
 - Comes from specs

• Accuracy

- The error w/r/t the truth
- Can be evaluated comparing with a “reference” or a different algorithm
 - In science “comparing different experiments”

• Target Accuracy

- The acceptable tolerance w/r/t the above
- Evaluated, for instance, w/r/t known error on the truth

	HIGH ACCURACY	MEDIUM ACCURACY	LOW ACCURACY
HIGH PRECISION	BARACK OBAMA WAS PRESIDENT FOR 70.128 HOURS	BARACK OBAMA WEIGHS AS MUCH AS 17.082 CATS	BARACK OBAMA IS 70.128 FEET TALL
MEDIUM PRECISION	MOST CATS HAVE 4 LEGS	BARACK OBAMA IS 6'1"	BARACK OBAMA HAS 4 LEGS
LOW PRECISION	MOST CATS HAVE LEGS	BARACK OBAMA HAS FEWER LEGS THAN YOUR CAT	BARACK OBAMA'S CAT HAS HUNDREDS OF LEGS

Cost of operations (in cpu cycles on x86_64)

https://www.agner.org/optimize/instruction_tables.pdf (icelake & zen5)

op	instruction	latency		inv throughput	
		s	d	s	d
+, -	ADD, SUB	3-4	3-4	0.5	0.5
==, <, >	COMISS CMP..	2,3	2,3	0.5-1	0.5-1
f=d, d=f, i=f, f=i	CVT..	3-7	3-7	0.5-1	0.5-1
, &, ^	AND, OR ...	1-2	1-2	0.25-0.5	0.25-0.5
*	MUL	5-4	5-4	0.5	0.5
a*b+c	FMA	4	4	0.5	0.5
/, sqrt	DIV, SQRT	11-20	13-29	3-12	4-24
1.f/ , 1.f/sqrt	RCP, RSQRT	4-5		0.5-1	
=	MOV	1,3,...	1,3,...	0.25,1,....	0.25,1,....

+ jump “overhead”
(mispredicted branches)

(does not vectorize
on intel?)

→ 350 from
main memory

What is faster?

continued fractions

Evaluating $R = \frac{x - \frac{7}{60}x^3}{1 + \frac{1}{20}x^2}$ costs 1 addition, 1 subtraction,
4 ~~x~~ multiplications, and 1 division.

We can rewrite R as follows:

$$\frac{(-7/20)x^3 + x}{((-7/20)x^3 - (7/3)x)} \bigg| \frac{(1/20)x^2 + 1}{(-7/3)x}$$

$$\begin{aligned} R &= -\frac{7}{3}x + \frac{\frac{10}{3}x}{1 + \frac{1}{20}x^2} = -\frac{7}{3}x + \frac{1}{\frac{1 + \frac{1}{20}x^2}{\frac{10}{3}x}} = -\frac{7}{3}x + \frac{1}{\frac{3}{200}x + \frac{1}{\frac{10}{3}x}} \\ &= -\frac{7}{3}x + \frac{200/3}{x + \frac{20}{x}}, 2 \text{ additions, 1 multiplication, 2 divisions} \end{aligned}$$

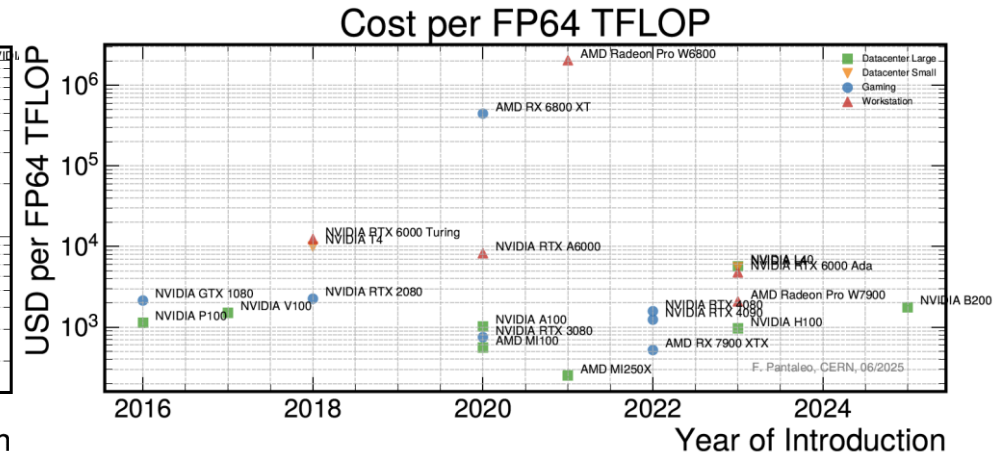
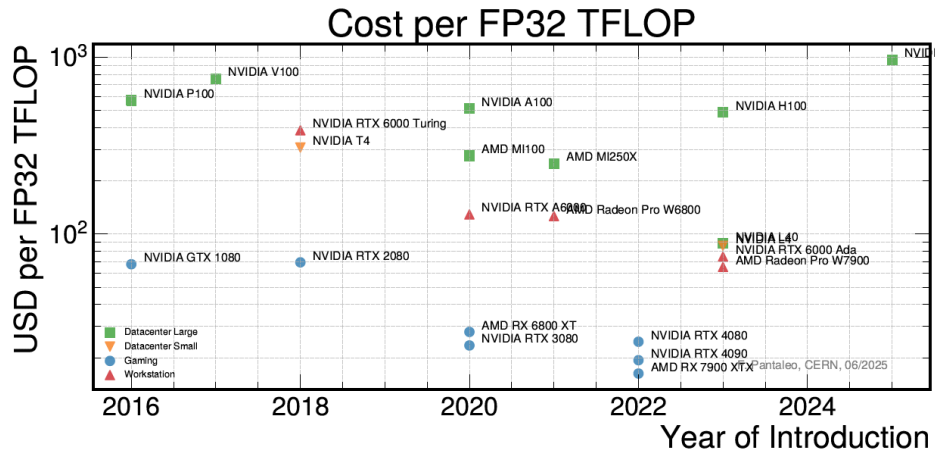
In literature is often quoted that “continued fractions” are faster because involve less “operations”

What do you think is faster on a modern architecture taking into account ILP and fma?

Source <https://homepages.math.uic.edu/~jan/mcs471/chebyshev.pdf>

GPU landscape:

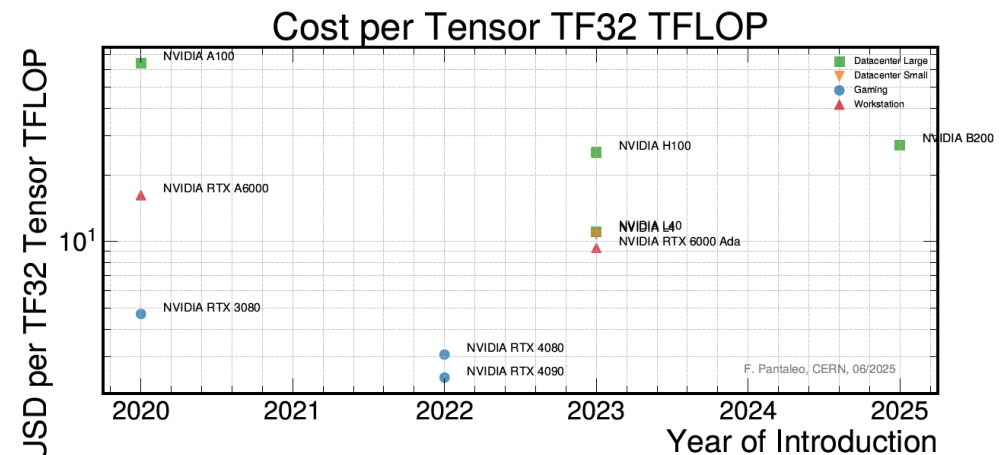
FP64 is 32 times slower than FP32 on affordable GPU



$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,15} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,15} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,15} \\ A_{15,0} & A_{15,1} & A_{15,2} & A_{15,15} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,15} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,15} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,15} \\ B_{15,0} & B_{15,1} & B_{15,2} & B_{15,15} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,15} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,15} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,15} \\ C_{15,0} & C_{15,1} & C_{15,2} & C_{15,15} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 FP16 or FP32

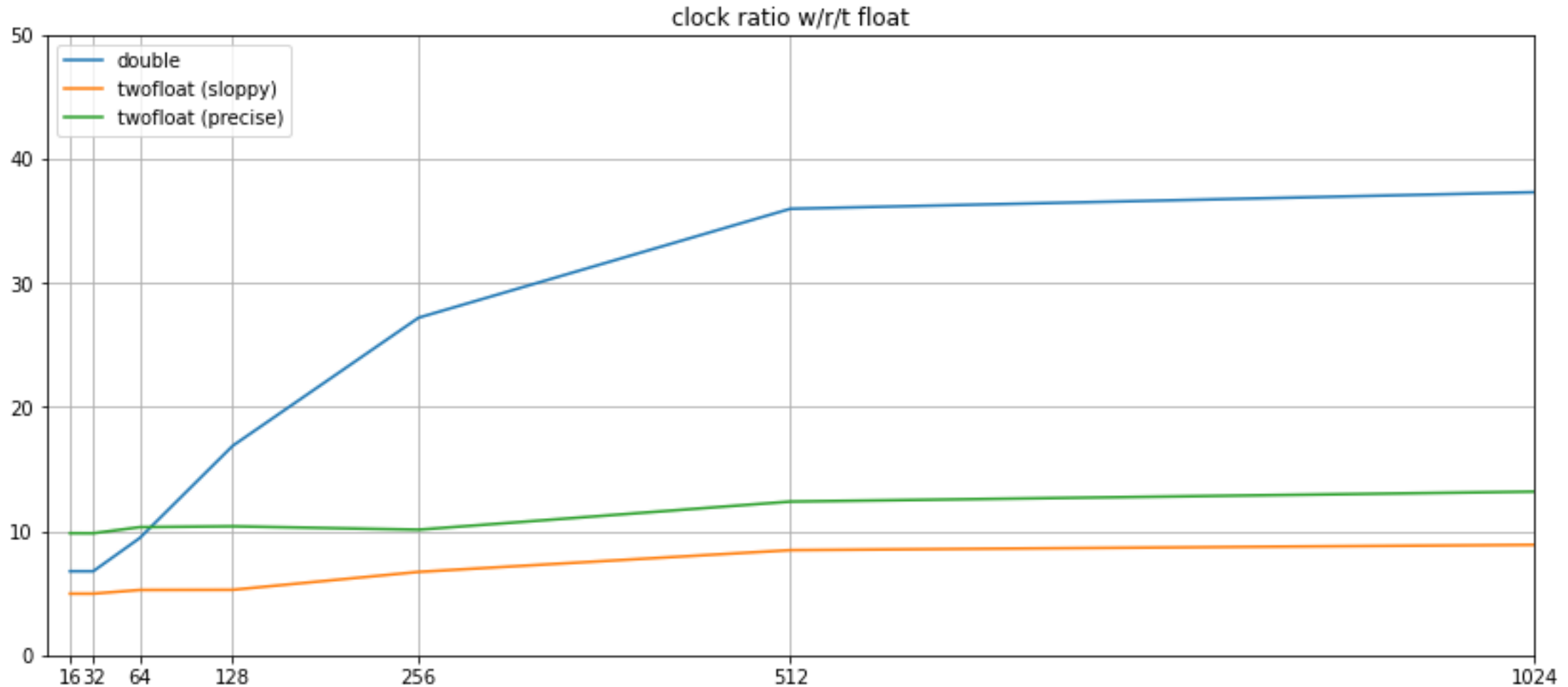
min rank for tensor is 16 (not 4)!



GPU (T4) performance

Brute-force inversion of 5x5 Pos-Def Matrix using Cholesky decomposition

33 + , 29 - , 126 * , 5 / , no sqrt (compiler optimization and fma will contract some)



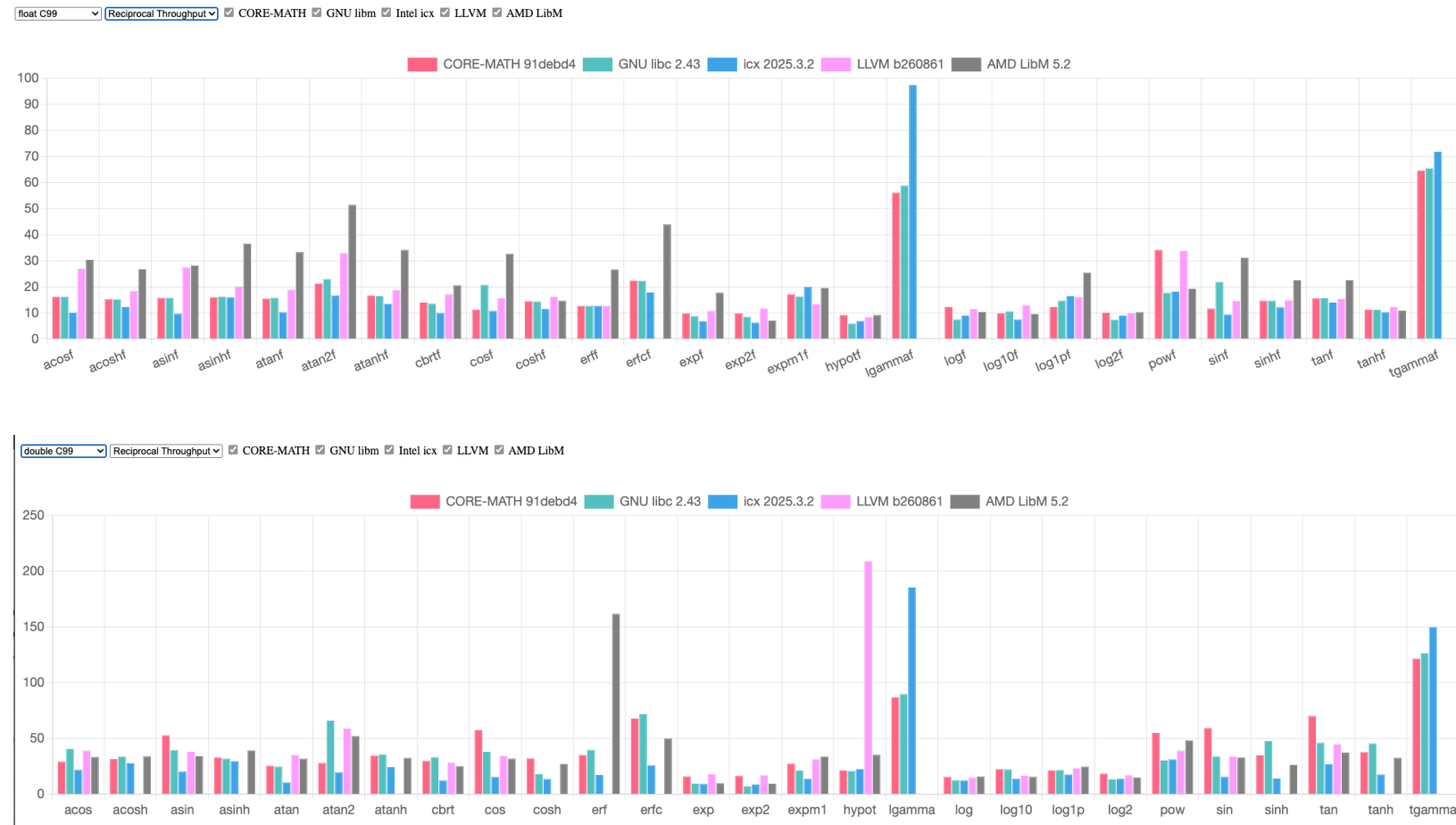
float vs double: summary

- In the old days: X87
 - Double faster than float!
- X86_64 (and ARM, PPC, RISC)
 - Scalar: float and double same speed (but div, sqrt, memory bandwidth)
 - Vector: float twice faster than double
- GPU: gaming and “Data Center”
 - Float 64-times faster than double
- GPU: HPC
 - Float twice faster than double
- GPU: AI
 - Optimized for low-precision tensor operations (GEMM can profit)

Elementary functions 1/throughput (in cycles)

(origin <https://core-math.gitlabpages.inria.fr/>)

Average number of clock cycles needed by CORE-MATH functions on an Intel Xeon Silver 4214, with GCC 15.2.0, compared to GNU libc, the Intel Math Library and LLVM libc (GNU libc was compiled with -O3 -march=native and linked statically, and GNU libc supports ermo by default, whereas CORE-MATH does not):



Precision vs Speed for elementary functions

- Log
 - 24 bit precision needs 8th degree polynomial
 - 16 bit precision needs 5th degree polynomial
 - 7 bit precision needs 2th degree polynomial

	2	3	4	5	6	7	8
e^x	6.8	10.8	15.1	19.8	24.6	29.6	34.7
$\sin(x)$	7.8	12.7	16.1	21.6	25.5	31.3	35.7
$\ln(1+x)$	8.2	11.1	14.0	16.8	19.6	22.3	25.0
$\arctan(x)$	8.7	9.8	13.2	15.5	17.2	21.2	22.3
$\tan(x)$	4.8	6.9	8.9	10.9	12.9	14.9	16.9
$\arcsin(x)$	3.4	4.0	4.4	4.7	4.9	5.1	5.3
$\sqrt{x}$	3.9	4.4	4.8	5.2	5.4	5.6	5.8

Number of significant bits of accuracy vs degree of minimax polynomial approximating the range [0..1].

Example: multiple scattering formula

```
double ms(double radLen, double m2, double p2) {  
    constexpr double amscon = 1.8496e-4; // (13.6MeV)**2  
    double e2 = p2 + m2;  
    double beta2 = p2/e2;  
    double fact = 1 + 0.038*log(radLen); fact *= fact;  
    double a = fact/(beta2*p2);  
    return amscon*radLen*a;  
}
```

Already an
approximation

Material density,
thickness, track angle
Known at percent?

```
float msf(float radLen, float m2, float p2) {  
    constexpr float amscon = 1.8496e-4; // (13.6MeV)**2  
    float e2 = p2 + m2;  
  
    float fact = 1.f + 0.038f*unsafe_logf<2>(radLen); fact /= p2;  
    fact *= fact;  
    float a = e2*fact;  
    return amscon*radLen*a;  
}
```

2nd order polynomial

Cash-Karp Runge-Kutta Step

3. A STRATEGY FOR DEALING WITH NONSMOOTH BEHAVIOR

The Runge-Kutta formula derived in the previous section has the special property that it contains imbedded solutions of all orders less than five. In addition, the formula has been designed so that the first five c_i values span the range $[0, 1]$ with reasonable uniformity, so that we have a very good chance of spotting bad behavior in f if it occurs. Our aim is to derive an automatic strategy that allows us to quit early, i.e., before all six function evaluations have been computed on the current step, if we suspect trouble, and to accept a lower order solution if appropriate.

We assume that we have computed a numerical solution y_{n-1} at the step point x_{n-1} and that for the current step, from x_{n-1} to $x_n = x_{n-1} + h$, all six function evaluations are computed so that solutions of all orders from 1 to 5 are available. (We guarantee this situation for the first step with $n = 1$). We denote the imbedded solution of order i at x_n by $y_n^{(i)}$, $1 \leq i \leq 5$, and define

$$\text{ERR}(n, i) = \|y_n^{(i+1)} - y_n^{(i)}\|^{1/(i+1)}, \quad \text{for } i \in 1, 2, 4. \quad (6)$$

We exclude the case $i = 3$ for two reasons. First, following the approach of Shampine et al. [15], we allow only a few different orders to be used, and we have chosen to allow orders 2, 3, or 5. Second, $\text{ERR}(n, 3)$ is of no use in predicting when to quit early since all six k_i 's are required before $y_n^{(4)}$ can be computed.

Suppose now that we were to accept the solution of order 5 at x_n . We wish to compute a suitable step length, $\bar{h}_4$, to be used in integrating from x_n to x_{n+1} using a 5(4) formula. A typical step-choosing strategy would compute $\bar{h}_4$ as

$$\bar{h}_4 = \frac{\text{SF} \times h}{E(n, 4)}, \quad \text{where } E(n, 4) = \frac{\text{ERR}(n, 4)}{\epsilon^{1/5}}. \quad (7)$$

Here ϵ is the local accuracy required (as specified by the user) and SF is a safety factor often taken to be 0.9. Similarly, if we were to accept either the second- or third-order solution at x_n , the steplengths $\bar{h}_1$, $\bar{h}_2$, respectively, that would be selected at the next step by our step-control algorithm would be

$$\bar{h}_i = \frac{\text{SF} \times h}{E(n, i)}, \quad \text{where } E(n, i) = \frac{\text{ERR}(n, i)}{\epsilon^{1/(i+1)}}, \quad i \in 1, 2. \quad (8)$$

$$0.9 * \text{step} / \text{pow}(\text{err} / \text{eps}, 0.2)$$

 RecoEcal/EgammaClusterProducers/src/EcalDigiSelector.cc

```

110 float energy1 = clus1.energy();
111 float theta1 = 2 * atan(exp(-1. * eta1));
112 float cluspt1 = energy1 * sin(theta1);

124 float energy1 = clus1.energy();
125 float theta1 = 2 * atan(exp(-1. * eta1));
126 float cluspt1 = energy1 * sin(theta1);

```

In experimental [particle physics](#), **pseudorapidity**, η , is a commonly used spatial [coordinate](#) representing the angle of a particle relative to the beam axis. It is defined as

$$\eta \equiv -\ln \left[\tan \left(\frac{\theta}{2} \right) \right],$$

where θ is the angle between the particle three-momentum $\mathbf{p}$ and the positive direction of the beam axis.^[1] Inversely,

$$\theta = 2 \arctan(e^{-\eta}).$$

As a function of three-momentum $\mathbf{p}$, pseudorapidity can be written as

$$\eta = \frac{1}{2} \ln \left(\frac{|\mathbf{p}| + p_L}{|\mathbf{p}| - p_L} \right) = \operatorname{arctanh} \left(\frac{p_L}{|\mathbf{p}|} \right),$$

<https://en.wikipedia.org/wiki/Pseudorapidity>

 Alignment/OfflineValidation/plugins/EopTreeWriter.cc

```

299 double dR;
300 // double Rec;
301 double theta1 = 2 * atan(exp(-eta1));
302 double theta2 = 2 * atan(exp(-eta2));
303 double cotantheta1;
304 if (cos(theta1) == 0)
305     cotantheta1 = 0;

```

 IOMC/ParticleGuns/src/FlatRandomOneOverPtGunProducer.cc

```

101 int PartID = fPartIDs[ip];
102 const HepPDT::ParticleData* PData = fPDGTable->particle(HepPDT::ParticleID(abs(PartID)));
103 double mass = PData->mass().value();
104 double theta = 2. * atan(exp(-eta));
105 double mom = pt / sin(theta);
106 double px = pt * cos(phi);
107 double py = pt * sin(phi);

```

$$p_x = p_T \cos \phi$$

$$p_y = p_T \sin \phi$$

$$p_z = p_T \sinh \eta,$$

which gives $|\mathbf{p}| = p_T \cosh \eta$.

 Geometry/HcalTowerAlgo/src/HcalDDDGeometryLoader.cc

```

109 HcalSubdetector subdet = detId.subdet();
110 double eta = 0.5 * (eta1 + eta2) * detId.zside();
111 double deta = (eta2 - eta1);
112 double theta = 2.0 * atan(exp(-eta));
113
114 // barrel vs forward
115 bool rzType = hcalCell.depthType();

```


Code pitfalls and opportunity for optimization

Language and compiler “rules”

- In essentially any language default FP arith is in double
 - Any other precision needs to be explicitly stated
 - One element in double and the whole expression will be evaluated in double
 - Promotion to double can be turn in a warning using “*-Wdouble-promotion*”
- C and C++ compilers (gcc, clang) by default will never reorder or transform expressions if the result may change
 - FMA is (currently) an exception
- C is a language born in the late `70
 - even C++ does not (potentially) break old code

Compiler option affecting FP Arith

<https://gcc.gnu.org/wiki/FloatingPointMath>

<https://clang.llvm.org/docs/UsersManual.html> - controlling-floating-point-behavior

- **-fno-math-errno**
 - Assume user ignore libc error flag
- **-fno-trapping-math**
 - Assume user ignore IEEE754 exceptions
- **-fno-signed-zeros**
 - Assume user do not distinguish sign of zeroes
- **-ffinite-math-only**
 - Assume code does not emit NaNs and Infs
- **-fassociative-math**
 - Allows free reordering of expressions
- **-freciprocal-math**
 - Allows transformation of divisions
- **-ffast-math**
 - All above and more

Controlling fma (contraction)

- gcc (and clang) flags
 - `-ffp-contract=off` (default is “fast”)
 - <https://stackoverflow.com/questions/43352510/difference-in-gcc-ffp-contract-options>
- cuda (nvcc) flags
 - `-fmad=false` (default is “true”)
 - <https://docs.nvidia.com/cuda/floating-point/index.html>
- fma function itself will in any case honored and will use the hardware instruction if supported

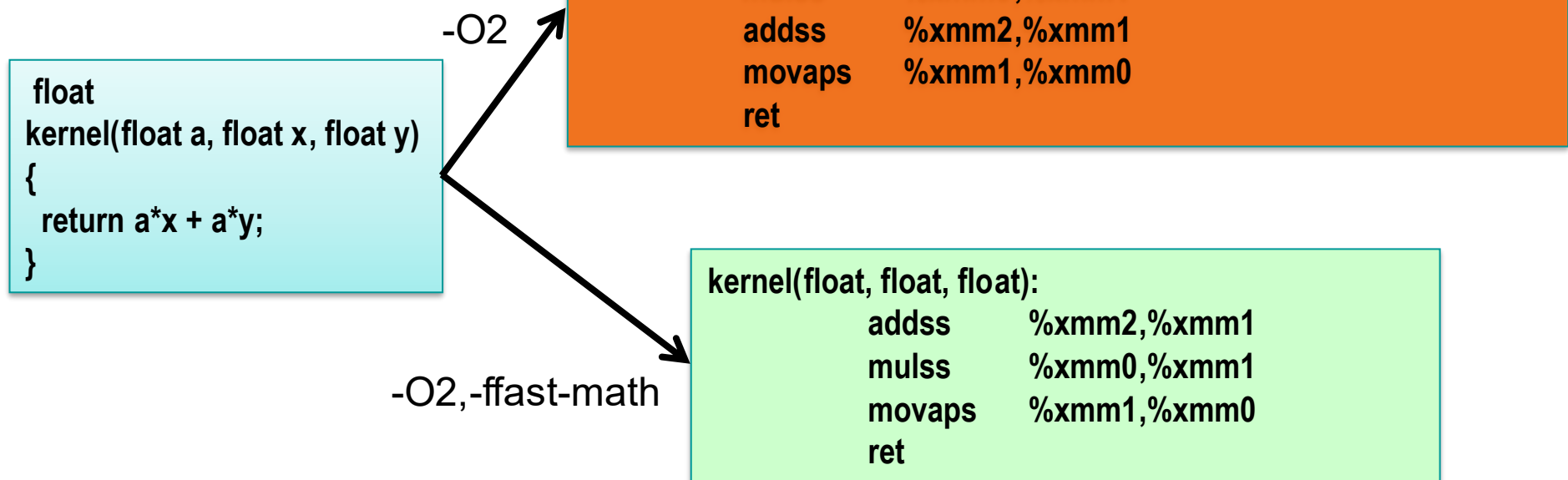
Inspecting generated code

`objdump -S -r -C --no-show-raw-insn -w kernel.o | less`

(on MacOS: `otool -t -v -V -X kernel.o | c++filt | less`)

Or use <http://gcc.godbolt.org>

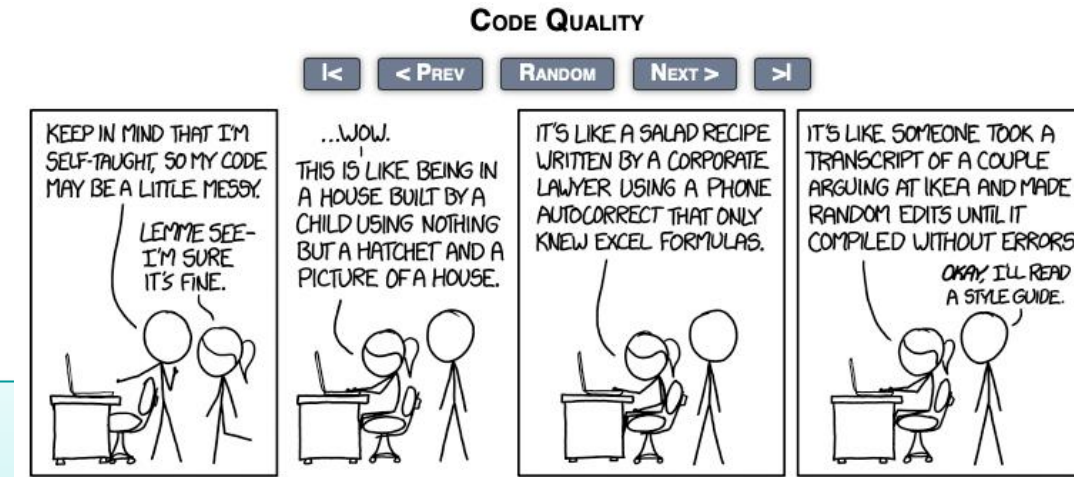
`c++ -S kernel.cc; less kernel.s`



Formula Translation: what was the author's intention?

```
// Energy loss and variance according to Bethe and Heitler, see also  
// Comp. Phys. Comm. 79 (1994) 157.  
//  
double p = mom.mag(); // sqrt(mom.x()*mom.x()+mom.y()*mom.y()+mom.z()*mom.z())  
double normalisedPath = fabs(p/mom.z())*radLen;  
double z = exp(-normalisedPath);  
double varz = (exp(-normalisedPath*log(3.)/log(2.))- exp(-2*normalisedPath));
```

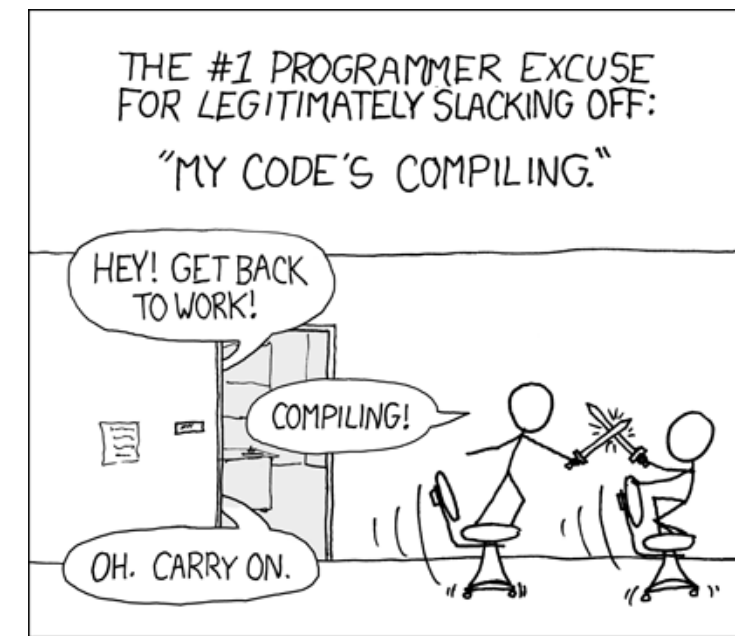
```
double pt = mom.perp(); // sqrt(mom.x()*mom.x()+mom.y()*mom.y())  
double j = a_i*(d_x * mom.x() + d_y * mom.y())/(pt*pt);  
double r_x = d_x - 2* mom.x()*(d_x*mom.x()+d_y*mom.y())/(pt*pt);  
double r_y = d_y - 2* mom.y()*(d_x*mom.x()+d_y*mom.y())/(pt*pt);  
double s = 1/(pt*pt*sqrt(1 - j*j));
```



Time for FP optimization!

- Do the math pencil on paper first
- Enable FMA
- Remove all unnecessary divisions and sqrt
- Enable “safe” compiler options
- Use double only when precision is required
 - On GPU: consider using augmented-arithmetic in single precision
- Consider to use custom polynomial approximations
 - When range is known and small
 - To match low target accuracy

EXERCISES



In C++

use godbolt (<https://godbolt.org/z/KjEjdzea9>)

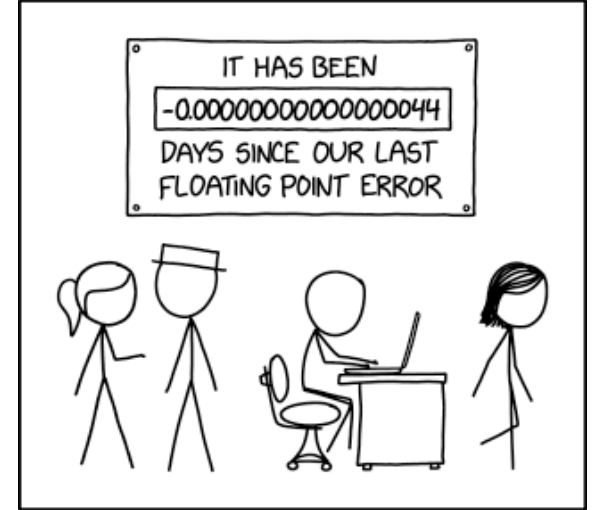
change compiler options for various compilers and see what happens

Try to optimize some code and see if it really improves

Anecdotes

- Floating point “disasters”

- <http://ta.twi.tudelft.nl/users/vuik/wi211/disasters.html>
- <https://www.iro.umontreal.ca/~mignotte/IFT2425/Disasters.html>
- <https://slate.com/technology/2019/10/round-floor-software-errors-stock-market-battlefield.html>
- https://medium.com/@sohail_saifii/the-floating-point-standard-thats-silently-breaking-financial-software-7f7e93430dbb
- <https://pavpanchekha.com/blog/casio-mathjs.html>

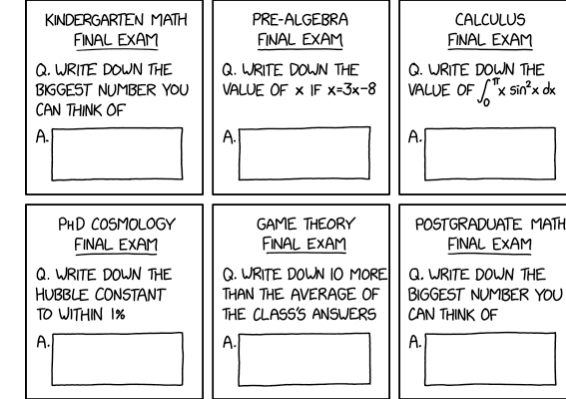


References

- What Every Computer Scientist Should Know About Floating-Point Arithmetic (1991) <https://dl.acm.org/doi/pdf/10.1145/103162.103163>
- Handbook of Floating-Point Arithmetic(2018) <https://link.springer.com/book/10.1007/978-3-319-76526-6>
- Elementary Functions (2016) <https://link.springer.com/book/10.1007/978-1-4899-7983-4>
- Floating-point arithmetic (2023) <https://www.cambridge.org/core/journals/acta-numerica/article/floatingpoint-arithmetic/287C4D5F6D4A43FBEEB1ABED2A405AAF>
- Low Precision (2025)
 - <https://www.siam.org/publications/siam-news/articles/mixed-precision-computing-high-accuracy-with-low-precision/>
 - https://arith2026.org/slides/keynote_1_Carson.pdf
- Bleeding edge
 - IEEE Arith Conference: <https://arith2026.org/archive.html>
 - FPTalks: <https://fptalks.org/>
 - NGT Workshop: <https://indico.cern.ch/event/1538409/>

Self Assessment

- How many significant bits does have a single precision FP number?
- How many single precision binary FP numbers exist between 2 and 3?
- Round 137.5 to 3 digits using default rounding
- Is the result of $137.5 - 82.4$ precise?
- If MAX is the largest representable single precision FP number: what's the result of $MAX + 0.1 - MAX$? And of $Max + Max/1024 - Max$?
- What is the result of $2.0 < NaN$ when compiled with `-ffinite-math-only`?
- What's the difference between $\sin(x)$ and $\sin\pi(x)$?
- When the "Table Maker Dilemma" occur?
- What is faster: a fma instruction or a division?
- How many fma instructions a modern processor can execute while a similar one is computing a single precision "exp" functions?

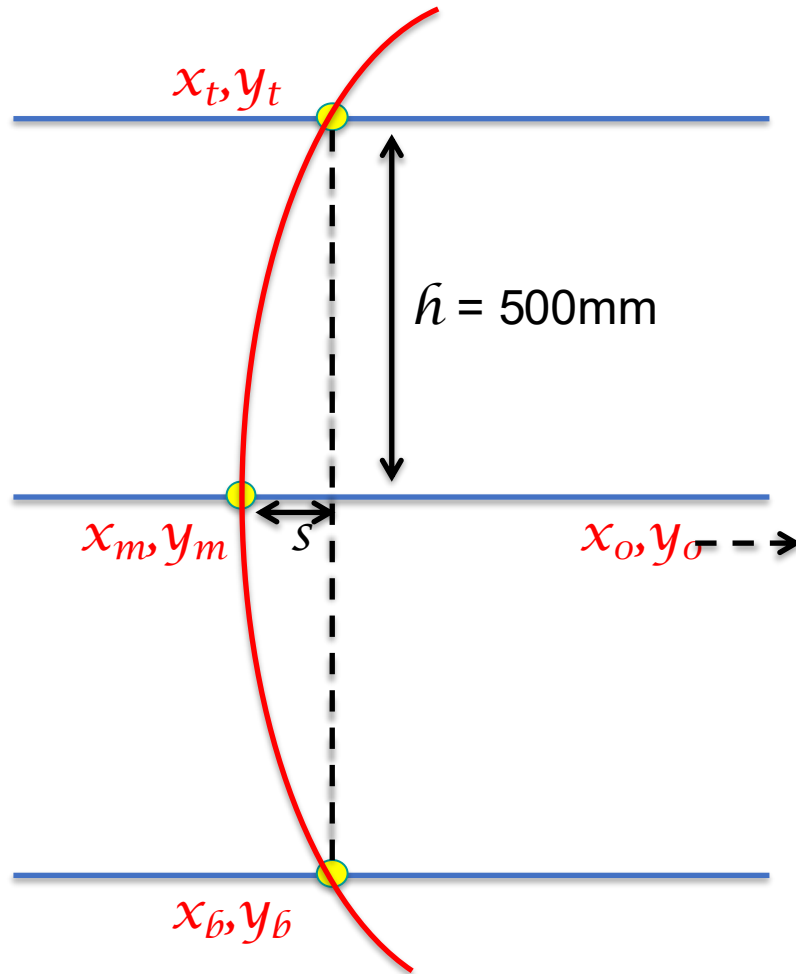


“additional” material

What precision is good enough?

bits	decimal	relative error	
8	2.3	3.9E-3	uint8 (byte)
12	3.3	5.0E-4	float16 (half)
16	4.8	1.5E-5	uint16 (short)
24	7.2	1.2E-7	float32 (float)
32	9.5	2.3E-10	uint32 (int)
53	16	1.1E-16	float64 (double)
64	19	5.4E-20	uint64 (long long)
113	34	1.9E-34	float128 (quad)

Example: detector of size 1 m and resolution 10 micron
(required accuracy: ~1micron?)



three measurement planes

Is single precision enough to represent

- geometric elements?
- Measurements?
- computed trajectory?

Is enough to perform computation?

Trajectory: circle with radius r
 s is the sagita, $\hat{h}$ the half-chord

$$r = (s^2 + \hat{h}^2) / 2s \sim \hat{h}^2 / 2s$$

$$\hat{h}^2 = 2sr - s^2$$

$$x_o = r + x_m$$

Circle equation and solutions

$$(x - \mathbf{x}_o)^2 + (y - \mathbf{y}_o)^2 = r^2$$

Compute $\mathbf{x}_t$ (x for $y = \hat{h}$)

$$x = \mathbf{x}_o - (r^2 - \hat{h}^2)^{1/2} \sim o(\text{zero})$$

Alternative formulation equivalent to change of coordinate centered at $\mathbf{x}_m, \mathbf{y}_m$

$$(x - \mathbf{x}_m)^2 + (y - \mathbf{y}_m)^2 - 2ar(x - \mathbf{x}_m) - 2br(y - \mathbf{y}_m) = 0$$

In our simplified case $a=1, b=0, \mathbf{y}_m=0$

$$(x - \mathbf{x}_m)^2 - 2(x - \mathbf{x}_m)r + \hat{h}^2 = 0$$

$$x = \mathbf{x}_m + r - (r^2 - \hat{h}^2)^{1/2} \sim \mathbf{x}_m \quad (\text{naïve: cancellation!!})$$

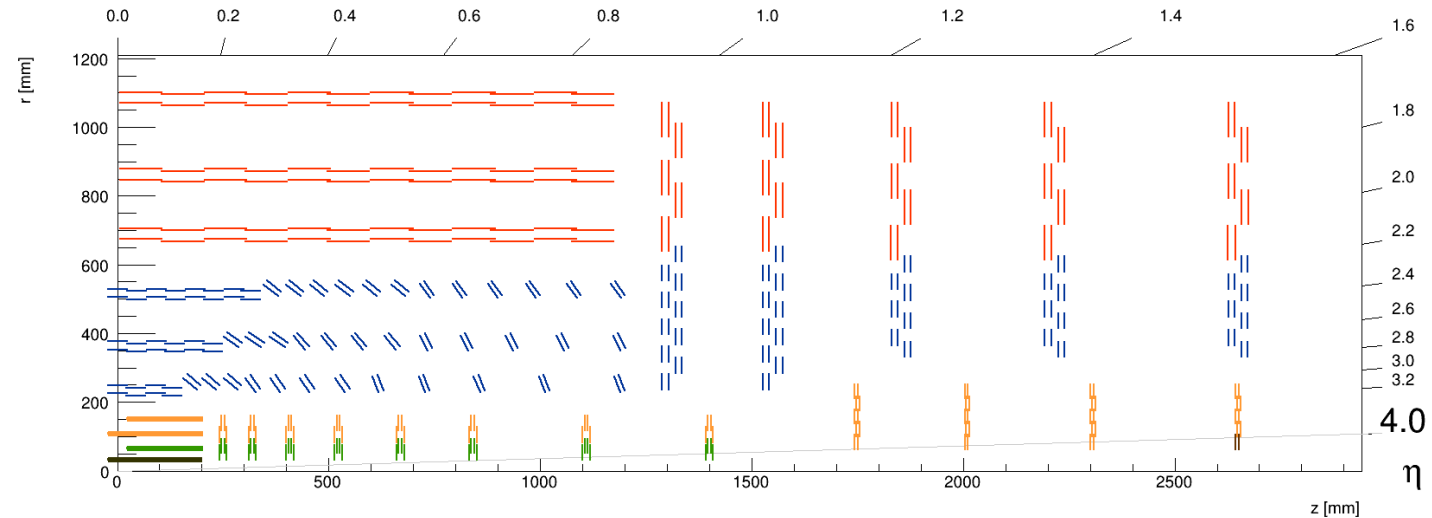
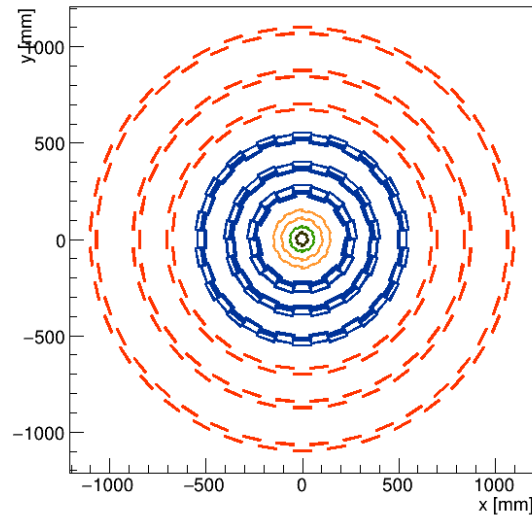
$$x = \mathbf{x}_m + \hat{h}^2 / (r + (r^2 - \hat{h}^2)^{1/2}) \sim \mathbf{x}_m + \hat{h}^2 / 2r \quad (\text{optimal})$$

A correct choice of algorithm can allow to use a reduced precision to obtain the same accuracy

- Faster
- Smaller memory footprint

Example: CMS tracker

https://cms-tklayout.web.cern.ch/cmssw/OT806_IT742/info.html



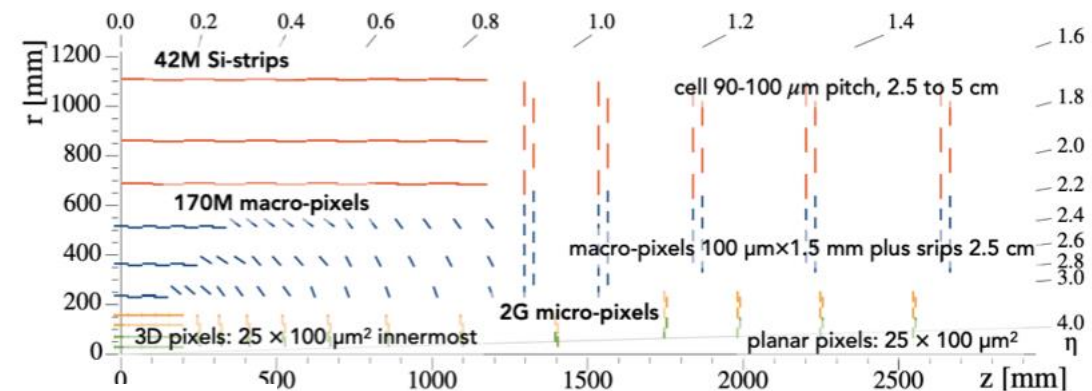
Ultimate single hit spatial resolution: (alignment error to be added)

IT: $\sigma_x \sim 3\mu$ $\sigma_z \sim 10\mu$ at $r < 15\text{cm}$ $|z| < 20\text{cm}$

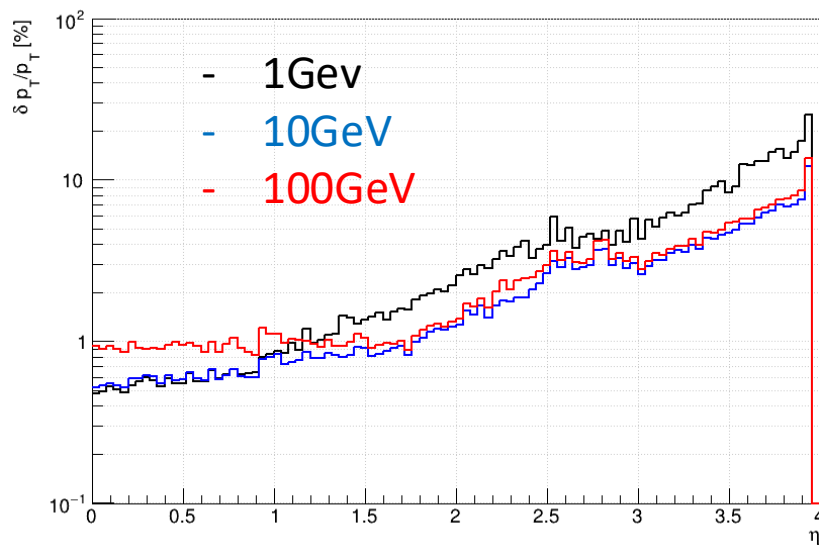
OT: $\sigma_x \sim 20\mu$ $\sigma_z \sim 425\mu$ at $r < 120\text{cm}$ $|z| < 130\text{cm}$

In both cases 19 bits (absolute: ie integer) are enough as precision

Ultimate track resolution

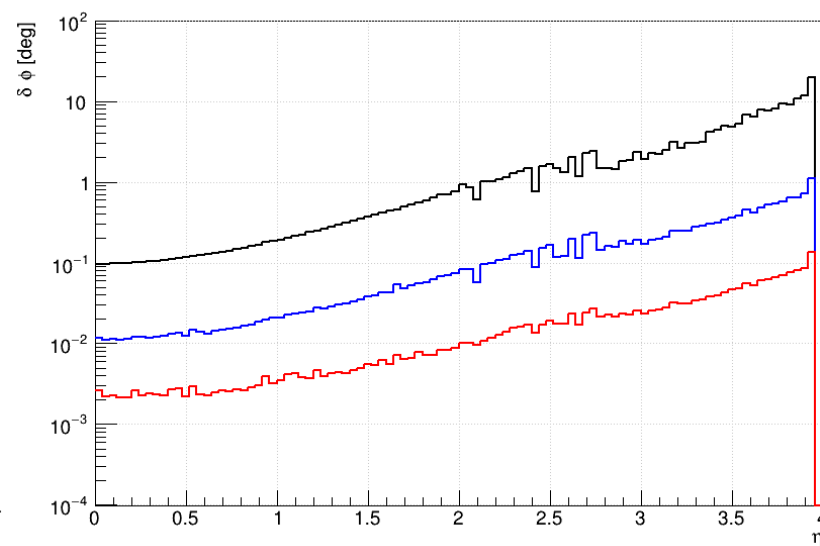


p_T resolution versus η - const P across η



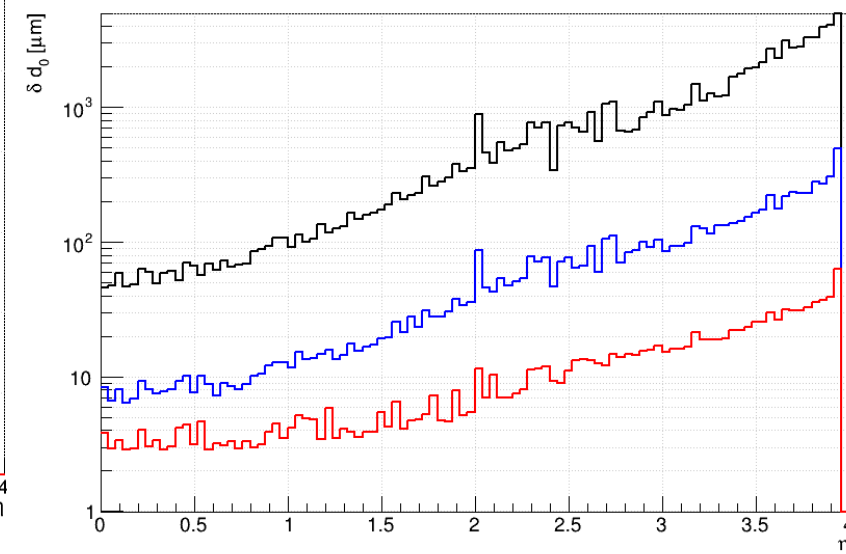
10 bits (relative)

Track azimuthal angle error - const P across η



18 bits absolute

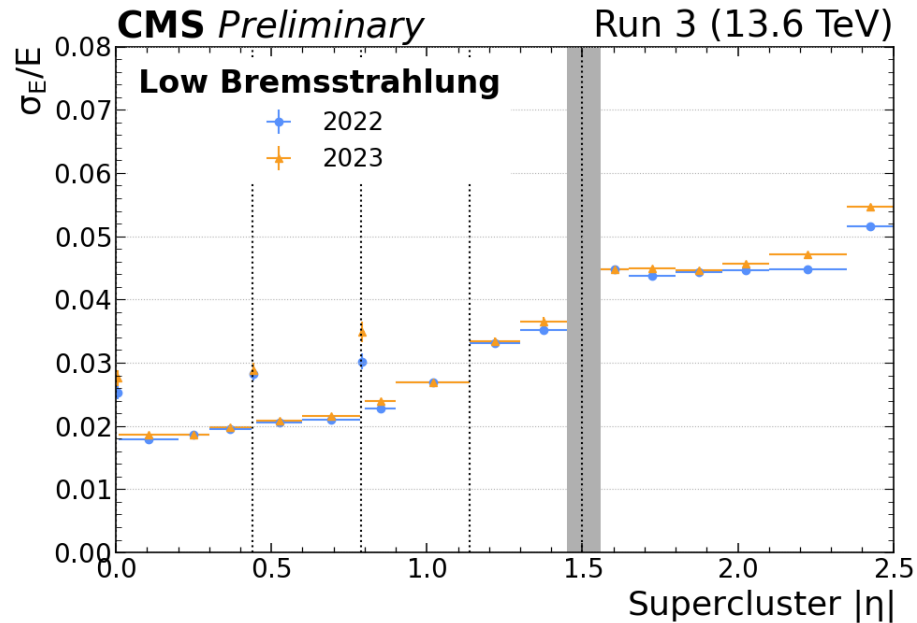
Transverse impact parameter error - const P across η



14 bits absolute (inside beam pipe)

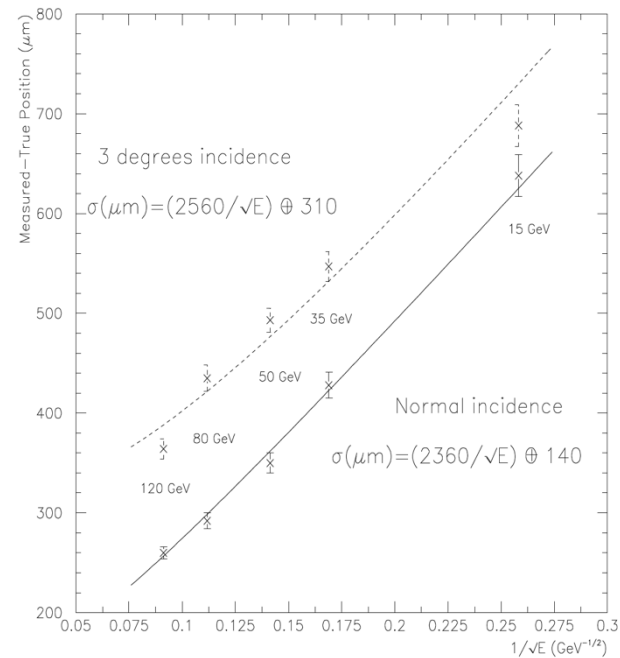
Example: CMS Electromagnetic Calorimeter

Energy Resolution



8 bits relative

Position Resolution – Neural Net Method



16 bits absolute

Example: cut in pt, phi, eta

```
inline float pt2(float x, float y) {return x*x+y*y;}
inline float pt(float x, float y) {return std::sqrt(pt2(x,y));}
inline float phi(float x, float y) {return std::atan2(y,x);}
inline eta(float x, float y, float z) { float t(z/pt(x,y)); return ::asinhf(t);}
inline float dot(float x1, float y1, float x2, float y2) {return x1*x2+y1*y2;}
inline float dphi(float p1, float p2) {
    auto dp=std::abs(p1-p2); if (dp>float(M_PI)) dp-=float(2*M_PI);
    return std::abs(dp);
};
```

```
if (pt(x[i],y[i])>ptcut) ...
```

```
if (dphi(phi(x[i],y[i]),phi(x[j],y[j]))<phicut) ....
```

Exercise in ptcut.cpp